2026년 AI 데이터기반지능행정 역량강화 사업

노코드 AI서비스 구현

Vibe Coding Basic

2026

행정안전부 NIA 한국지능정보사회진흥원 NATIONAL INFORMATION SOCIETY AGENCY

노코드 AI서비스 구현[커리큘럼]

시간	모듈	세부 학습 내용	도구
DAY 1 · TECH TASTING & REVERSE ENGINEERING — 기술 체험과 사례 해부 (실습 중심)			
1	M1. 바이브 코딩 패러다임 + 코딩에이전트 개관	바이브 코딩의 등장과 비개발자의 새로운 역할. 거시 소버린 AI / 미시 바이브 코딩 두 축의 의미. 코딩에이전트(Antigravity·Claude Code 등)의 작동 위치와 본 과목에서 다룰 범위. C05(기획) → C06(구현)의 연결과 「폐쇄망 먼저, 클라우드 마지막」 흐름 안내. 민첩한 AI 8가지 기술 중 본 과목 커버 범위 명시.인트로 only — 2일 프로젝트 목표 1페이지(문제·솔루션·기술 스택) 작성	강의 · 워크숍
2	M2. 코딩에이전트로 Python 업무자동화	개념·원리: 코딩에이전트의 반복 수정 루프, 외부 LLM이 짜준 .py를 내부망에서 실행하는 폐쇄망 안전 워크플로, 표준 라이브러리(os·shutil·pandas·pdfplumber·python-docx)의 역할 분담.실습 1. 특정 경로의 PDF 파일에서 텍스트·표를 추출해 엑셀로 옮기는 스크립트를 받아 실행 / 실습 2. 폴더 내 다수 CSV를 통합 엑셀 보고서로 자동 변환 / 실습 3. 워드 양식을 수신자별 맞춤으로 일괄 생성	Python Antigravity Claude
3	M3. HTML/CSS/JS 단일파일 웹앱	개념·원리: 단일 HTML 파일의 구조(head·body·script·style), CDN·인라인 라이브러리 반입, 브라우저 내부에서 완결되는 도구의 의미와 한계. LLM에게 단일 파일 코드를 요청하는 프롬프트 패턴.실습 1. 부서 소개 정적 페이지를 단일 HTML로 작성 후 더블클릭 실행 / 실습 2. 단일 파일 계산기·타이머·체크리스트 중 1종 선택 구현	HTML/CSS/JS Claude ChatGPT
4	M4. 클라이언트 영속성 (localStorage · IndexedDB)	개념·원리: 브라우저 저장소의 작동 방식과 한계, localStorage(키-값) vs IndexedDB(구조화 데이터)의 차이, 서버 없이도 영속 데이터를 갖는 도구의 패턴.실습 1. 메모/할일 앱 / 실습 2. 즐겨찾기·북마크 관리 도구 / 실습 3. 폼 입력 임시저장 기능	localStorage IndexedDB
5	M5. 엑셀 VBA × LLM 어시스턴트	개념·원리: VBA가 엑셀과 결합되는 구조(이벤트·매크로·UDF), LLM에게 VBA 매크로를 요청하는 프롬프트 패턴, 엑셀 기반 부서 자동화의 가치 재발견.실습 1. 일일 매출/통계 보고서 자동 생성 매크로 / 실습 2. 받은 이메일 자동 분류·요약 매크로(Outlook VBA)	VBA Excel Outlook
DAY 2 · PLANNING & IDEATION — 기획과 아이디어 발굴 (실습 중심)			
1	M6. Ollama + 레거시 폐쇄망	개념·원리: Ollama로 1B~2B 로컬 SLM을 CPU에서 구동하는 방식, 엑셀·워드에서 로컬 SLM을 호출하는 구조, 폐쇄망 부서가 외부 API 없이 LLM 능력을 갖는 가치.실습 1. 엑셀 셀에서 Ollama로 사내 문서 요약 / 실습 2. 워드 매크로에서 Ollama로 어투·맞춤법 교정	Ollama Excel VBA Word VBA
2	M7. 클라우드 노코드 풀스택 비교	개념·원리: Lovable·Bolt·v0·Replit Agent·Base44 각 도구의 작동 방식과 강·약점, 자동 DB 연결의 의미, 도구 선정 기준.실습. 동일 요구사항(부서 게시판 1개 화면)을 두 도구에 동시 제작하고 결과 비교표 작성	Lovable Bolt v0 Replit Base44
3	M8. Lovable + Supabase 자동연동	개념·원리: Lovable이 코드 생성과 Supabase DB·Auth·Storage 연결을 어떻게 묶는가, 노코드 풀스택의 데이터 흐름, 노코드 도구가 만든 코드를 직접 살펴보는 가치.실습 1. 부서 게시판(회원·글·댓글) / 실습 2. 결재 신청 양식 + 승인 흐름 / 실습 3. 일정·회의실 예약 시스템	Lovable Supabase
4	M9. 디자인 → 코드 파이프라인 (Stitch)	개념·원리: Google Stitch가 자연어·이미지에서 UI를 만드는 방식, AI Studio·v0가 디자인을 코드로 변환하는 흐름, 디자인-투-코드 파이프라인이 비개발자에게 의미하는 것.실습 1. Stitch로 모바일 앱 화면 디자인 → AI Studio로 HTML 변환 / 실습 2. 대시보드 화면 스케치 → v0 컴포넌트 생성	Stitch Google AI Studio v0
5	M10. 인증·권한 기초	개념·원리: 웹 인증의 기본 구조(세션 vs 토큰), Auth(Email·OAuth) 흐름, Access Token·환경변수 관리, Row-Level Security(RLS)의 개념과 필요성.실습 1. Supabase Auth로 가입·로그인 적용 / 실습 2. 권한별 페이지 분리(일반/관리자) 구현	Supabase Auth JWT
6	M11. GitHub + Netlify/Vercel 배포	개념·원리: Git 기본 흐름(add·commit·push), GitHub 저장소 연동, Netlify·Vercel·GitHub Pages의 차이, CI/CD의 개념과 환경변수 관리.실습 1. Lovable 프로젝트를 GitHub→Netlify로 자동 배포 / 실습 2. 환경변수로 API 키를 안전하게 관리	GitHub Netlify Vercel
7	M12. 종합 워크숍 — 전체 종합 실습	전체 누적 도구를 결합하여 부서 단위 통합 서비스를 완성함. Python 자동화 스크립트 또는 VBA 매크로 1종 + 클라우드 풀스택 웹앱 1종을 결합하여 시연. 발표 시나리오 작성과 시연 리허설. 가져갈 산출물 점검과 README 작성.종합실습 + 발표. 폐쇄망 자동화 1종 + 클라우드 풀스택 웹앱 1종 결합 시연 + README + 다음 단계 계획서	

M1. 바이브 코딩 패러다임 + 코딩에이전트 개관

서로 넘보기 힘들었던 직업군

아이디어는 있는데 이걸 어떻게
개발하지. 개발자 월급은?
개발자에게 뭐라고 설득하지?

개발자

기획자

개발은 나의 일이 아니야..

서로 넘보기 힘들었던 직업군

현장에서 정말 필요한 서비스는 뭐지
사용자는 무슨 생각을 할까
서비스를 오버뷰할수 있었으면
좋겠는데

개발자

원하는대로 개발하면 되지
저 프로그랭언어는
안배웠는데...

기획자

“개발자도 기획자가 되는 시대”

이제 개발자란 단순히 남이 시키는 대로 코딩만 하는 존재가 아닙니다.
생성형 AI와 다양한 자동화 도구 덕분에
개발자 역시 ‘수동적인 기술자’에서 벗어나,
서비스를 ‘직접 기획하고 창조하는 역할’로 변화하고 있습니다.

“요즘 개발자는 ‘코딩만 잘하는 사람’이 아니라,아이디어를 만들고, 시장을 읽고, 제품을 직접 설계하는 ‘기획자+개발자’로 변신하고 있습니다.생성형 AI가 이런 변화를 가속화시키고 있죠.”

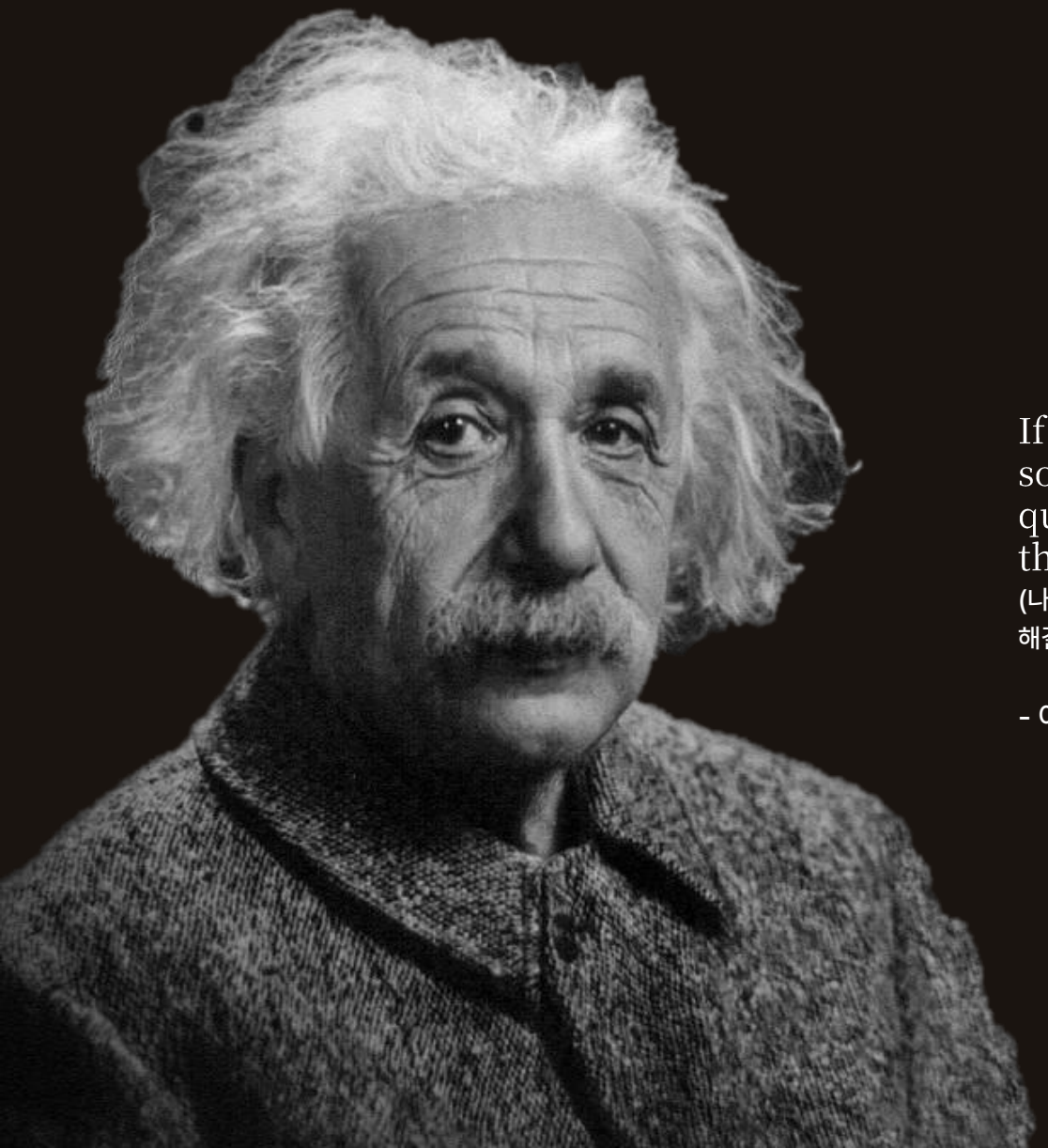
“비개발자도 풀스택 개발자로”

이제 개발 경험이 전혀 없는 기획자조차도오픈소스, 생성형 AI,
노코드 도구만 잘 활용하면'풀스택 개발'(기획→구현→배포→운영)까지
스스로 해낼 수 있는 시대가 되었습니다.

- 예전에는 '개발=개발자의 영역'이었지만
- 이제는 기획자도 AI의 힘을 빌려
- 직접 서비스를 만들고, 자동화하고, 배포까지 가능

예전에는 개발자에게 부탁해서 서비스 만들던 기획자가
이제는 GPT, 오픈소스, 노코드 플랫폼 덕분에혼자서
아이디어를 실현하고, 직접 배포하는 풀스택 개발자가 될 수 있습니다.

이 수업이 바로 그 실전 경험을 제공하는 강의입니다!



If I had an hour to solve a problem and my life depended on the solution, I would spend the first 55 minutes determining the proper question to ask, for once I know the proper question, I could solve the problem in less than 5 minutes.

(나에게 1시간이 주어진다면 문제가 무엇인지 정의하는 데 55분의 시간을 쓰고,
해결책을 찾는 데 나머지 5분을 쓸 것이다.)

- 아인슈타인(Albert Einstein)

M1. 바이브 코딩 패러다임 + 코딩에이전트 개관

바이브 코딩이란 무엇인가

바이브 코딩

코드를 직접 타이핑하지 않고 LLM·코딩에이전트에게 자연어로 의도를 전달해 코드를 받아 실행하는 작업 방식. 비개발자가 도구·서비스를 직접 만들 수 있게 된 결정적 전환점.

구분	전통 개발	바이브 코딩
주역할	코드 작성	요구사항 설계 + 결과 검수
입력	문법·라이브러리 암기	자연어 의도·맥락
반복 단위	줄 단위 디버깅	대화 단위 수정
도구 의존	IDE·문법	LLM·에이전트
시간 단위	주·월	분·시간

코딩에이전트의 등장 — 채팅 LLM과의 차이

채팅 LLM은 「코드 한 덩어리를 받아오는」 방식. 코딩에이전트는 「프로젝트 안에 들어와서 직접 파일을 만들고 고치는」 방식.

	채팅 LLM (ChatGPT · Claude · Gemini)	코딩에이전트 (Antigravity · Claude Code · OpenCode · Codex)
작동 위치	웹 대화창	IDE·터미널 안
출력	코드 블록 1덩어리	다중 파일 변경
컨텍스트	사람이 매번 붙여넣음	프로젝트 전체 자동 인식
반복 수정	사람이 복붙해서 다시 질문	에이전트가 스스로 고쳐가며 진행
적합 작업	단일 HTML·짧은 스크립트	다중 파일·반복 디버깅·풀스택

M1. 바이브 코딩 패러다임 + 코딩에이전트 개관

바이브 코딩이란 무엇인가

코드를 못 짜는 게 문제가 아니라, 무엇을 만들지 모르는 게 문제다. 본 과목은 「무엇을 만들지 정확히 말하고, 받은 결과를 검수하는」 능력을 12모듈 동안 반복적으로 훈련한다.

바이브 코딩 작업 루프 — 3단계 반복

요구사항 설계 → 코드 생성·수신 → 검수·실행·피드백 세 단계를 짧게 반복한다. 한 번에 완성하지 않고, 작은 단위로 받아 확인하고 다음 요청을 더 정확하게 만드는 것이 핵심.

- **1단계 — 요구사항 설계**: 무엇을, 왜, 어떤 조건으로 만들 것인지를 자연어로 정리
- **2단계 — 코드 생성·수신**: LLM 또는 코딩에이전트에게 전달하고 결과 받기
- **3단계 — 검수·실행·피드백**: 받은 코드를 읽고 실행해서 작동 확인, 차이를 다음 요청에 반영

비개발 실무자가 갖춰야 할 4가지

채팅 LLM은 「코드 한 덩어리를 받아오는」 방식. 코딩에이전트는 「프로젝트 안에 들어와서 직접 파일을 만들고 고치는」 방식.

#	능력	의미
01	요구사항 명료화	모호함을 줄이는 글쓰기 — LLM에 정확히 의도 전달
02	결과 검수	받은 코드가 무엇을 하는지 최소한 읽어내는 능력
03	도메인 지식	부서 업무·맥락을 정확히 설명할 수 있는 깊이
04	도구 감각	같은 문제를 단일 HTML·Python·클라우드 중 무엇으로 풀지 판단

폐쇄망 부서에 특히 강력한 이유

외부 LLM에 코드를 받아 내부망에서 실행하는 비대칭 구조. 데이터를 외부로 보내지 않으면서 외부의 지식만 빌려 쓴다. 보안 검토 범위가 「외부에 무엇을 보냈는가」에서 「실행될 코드가 무엇을 하는가」로 좁아진다.

M1. 바이브 코딩 패러다임 + 코딩에이전트 개관

바이브 코딩 패러다임 + 코딩에이전트 개관

이 과목에서 다루는 것

거시·미시 두 축에서 본 과목의 위치를 잡고, 비개발 실무자가 LLM·코딩에이전트와 협업하는 새로운 작업 방식을 개관한다.
C05(기획) → C06(구현) → C08(인프라) 트랙 흐름과 「폐쇄망 우선」 원칙을 확인한 뒤, 12모듈 로드맵을 받아본다.
마지막에 본인의 2일 프로젝트 목표를 1페이지로 정의하는 워크숍으로 마무리한다.

- 바이브 코딩 패러다임이 비개발 실무자에게 무엇을 가능하게 하는지 설명할 수 있다
- 채팅 LLM과 코딩에이전트의 차이를 이해하고 본 과목 모듈별 적합한 도구를 선택할 수 있다
- 본 과목이 트랙 전체에서 차지하는 위치(C05·C08과의 연결)를 파악한다
- 본인 부서의 2일 프로젝트 목표를 「문제·솔루션·기술 스택」 1페이지로 정의한다

#	능력	의미
채팅 LLM	M3·M5의 가벼운 코드 받기	ChatGPT · Claude · Gemini
코딩에이전트	M2 Python 자동화 · 다중파일 작업	Antigravity · Claude Code
클라우드 노코드 풀스택	M7~M11 웹앱 구현	Lovable · Bolt · v0 · Replit · Base44
디자인-투-코드	M9 UI 목업	Stitch · AI Studio
로컬 SLM	M6 폐쇄망 LLM	Ollama

M2. 코딩에이전트로 Python 업무자동화

M2. 코딩에이전트로 Python 업무자동화

파이썬 – Anaconda

아나콘다(Anaconda) 소개 https://www.anaconda.com/download

아나콘다는 Numpy, Pandas, Jupyter Notebook, IPython, scikit-learn, matplotlib, seaborn, nltk 등
파이썬 버전 문제를 해결한 라이브러리 패키지를 몽땅 쉽게 설치할 수 있게 만든 것

① 다양한 라이브러리와 패키지 관리의 어려움

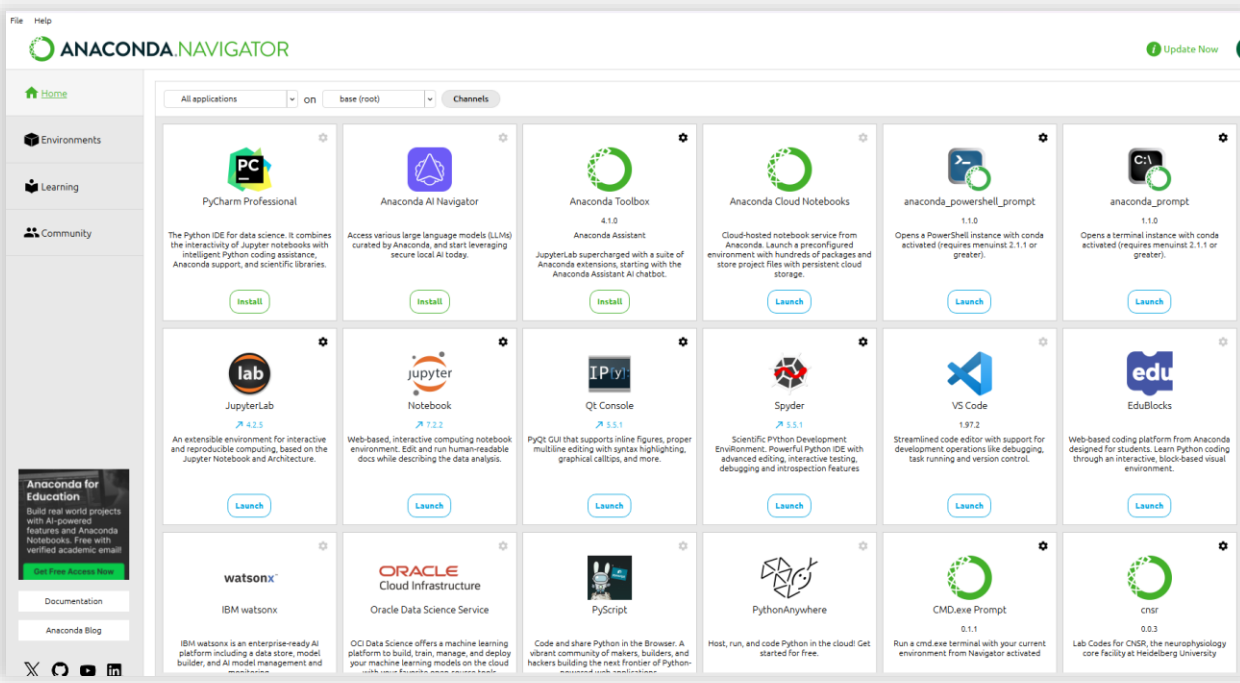
- 데이터 분석 & 머신러닝에 필요한 패키지(pandas, numpy, scikit-learn 등)가 너무 많음
- pip install로 하나씩 설치하는 것이 번거롭고, 버전 충돌 문제 발생 가능
- 아나콘다는 필수 패키지를 한 번에 설치하고 관리할 수 있도록 패키지 관리자(conda)를 제공

② 가상 환경(virtual environment) 쉽게 관리

- 프로젝트마다 다른 라이브러리 버전이 필요할 수 있음
- conda를 사용하면 서로 다른 환경을 격리하여 문제 없이 실행 가능
- (예: Python 3.8 프로젝트, Python 3.11 프로젝트 각각 독립 실행 가능)

③ 데이터 과학 & AI 연구를 쉽게 시작하도록 지원

- 주피터 노트북(Jupyter Notebook), 스파이더(Spyder) 등 개발 도구 내장
- 설치 후 바로 실행 가능 (환경 설정이 간편)



M2. 코딩에이전트로 Python 업무자동화

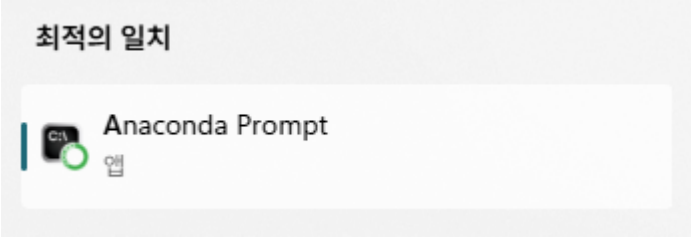
파이썬 – Anaconda

아나콘다(Anaconda) 설치방법

아래 사이트 접속 → 최신 버전 다운로드 (Python 3.x) → https://www.anaconda.com/products/distribution

Windows용 설치파일(.exe) 다운로드 (64-bit installer 권장)

단계	설명
사용자 대상 설치	"Just Me" 선택 (권장)
설치 경로	기본 경로 (C:\Users\사용자명\anaconda3) 그대로 사용
✓ Add Anaconda to my PATH environment variable	반드시 체크! (설치 후 환경변수 자동 등록됨)
✓ Register Anaconda as my default Python	체크 추천 (기본 Python을 Anaconda로 설정)



[아나콘다 가상환경 설정]
conda create -n openapi python=3.10
conda activate 가상환경명
pip install pandas numpy matplotlib jupyter ipykernel
python -m ipykernel install --user --name openapi --display-name "Python (가상환경명)"

M2. 코딩에이전트로 Python 업무자동화

파이썬 – Anaconda

아나콘다(Anaconda) 실행방법

방법 1:
Anaconda Prompt로 폴더 이동 후 실행

❶ Anaconda Prompt 실행 : (시작 메뉴 → Anaconda Prompt 검색)

❷ cd명령어로 원하는 경로로 이동 cd /d D:\project\ai-openapi

❸# 가상환경 활성화 : conda activate openapi

❹ # Jupyter Notebook 실행 : jupyter notebook

방법 2:
폴더 주소창에 cmd 입력 → 그 위치에서 명령 실행

M2. 코딩에이전트로 Python 업무자동화

파이썬 – python.org

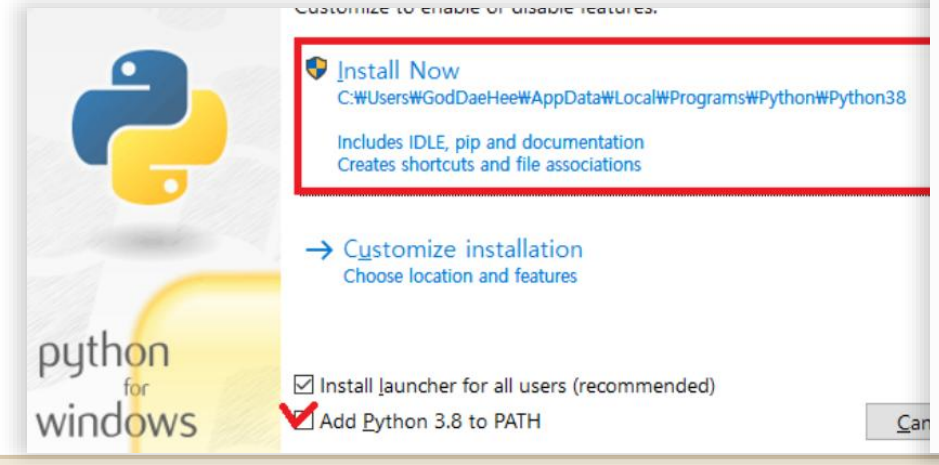
파이썬 설치 설치방법

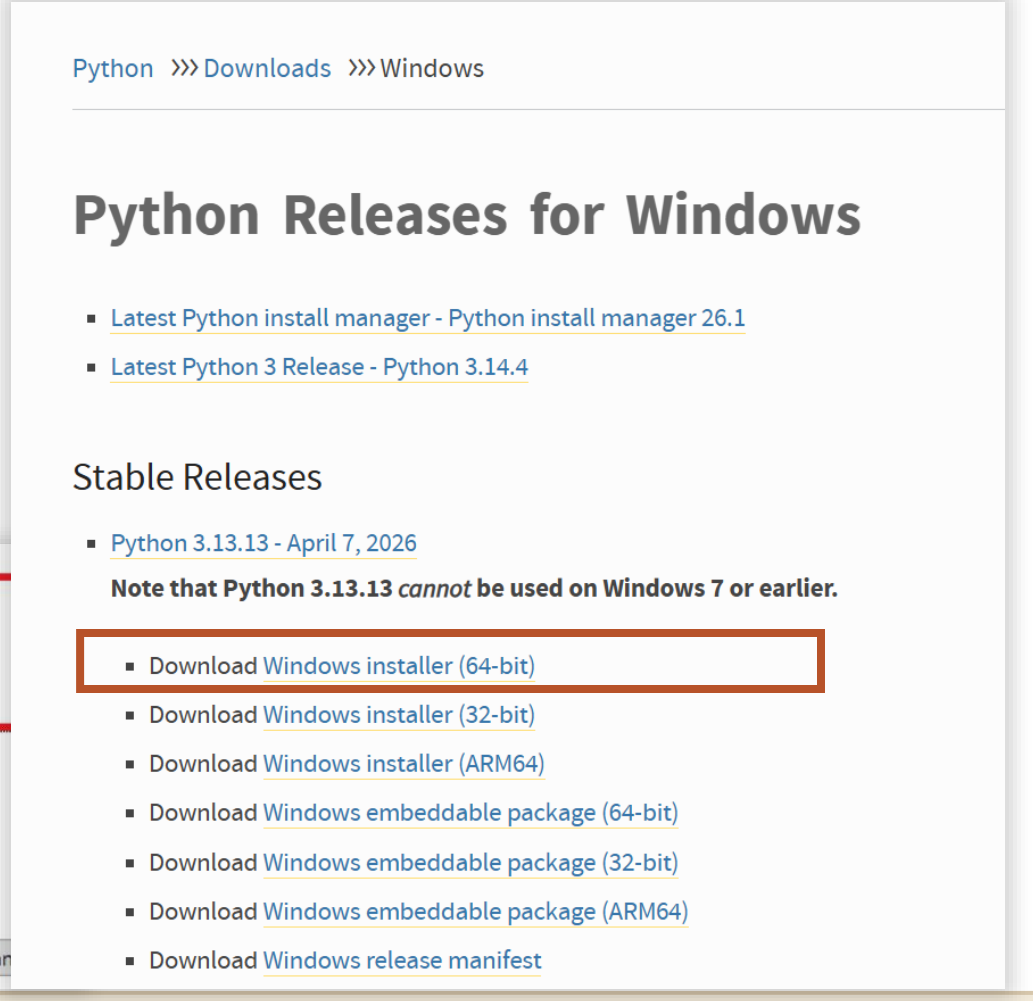
python.org에 접속하면 상단 메뉴에 "Downloads"가 있고,
Windows 사용자에게는 자동으로 최신 안정 버전(Stable release)의 다운로드 버튼이 표시

설치

첫째, 화면 하단의 **"Add python.exe to PATH"** 체크박스입니다. 이것을 체크하지 않으면 명령 프롬프트에서 python 명령어를 인식하지 못합니다.

설치가 완료되면 마지막 화면에서 "Disable path length limit" 링크가 나타나는데, 이것도 클릭해 두는 것이 좋습니다. Windows의 260자 경로 제한을 해제하는 것으로, 이후 패키지 설치 시 경로 오류를 예방합니다.





M2. 코딩에이전트로 Python 업무자동화

파이썬 – python.org

파이썬 실행방법

Windows 키를 누르고 cmd를 입력해 명령 프롬프트를 엽니다. 그 다음 아래 명령어를 순서대로 실행합니다.

python –version
pip --version

만약 'python'은(는) 내부 또는 외부 명령...이 아닙니다라는 메시지가 나오면 PATH 설정이 누락된 것이므로, 파이썬을 다시 설치하면서 "Add python.exe to PATH"를 체크하거나, 시스템 환경 변수에 수동으로 Python 경로를 추가해야 합니다.

pip install pandas numpy matplotlib seaborn

관공서 PC는 보안 정책으로 인해 소프트웨어 설치가 제한되는 경우가 많습니다. 사전에 정보보안 담당자에게 python.org 설치 파일 실행 허용을 요청하거나, 교육 당일 사용할 PC에 미리 설치해 두는 것이 필요합니다. pip 명령어도 외부 네트워크 접근이 필요하므로, 방화벽에서 pypi.org 도메인이 차단되어 있으면 패키지 설치가 안 될 수 있습니다. 이 경우 오프라인 설치용 .whl 파일을 미리 준비하는 방안도 교안에 언급해 두시면 좋겠습니다.

방법1:

인터랙티브 모드(즉석 실행)

명령 프롬프트에서 python만 입력하면 >>> 프롬프트가 나타납니다.

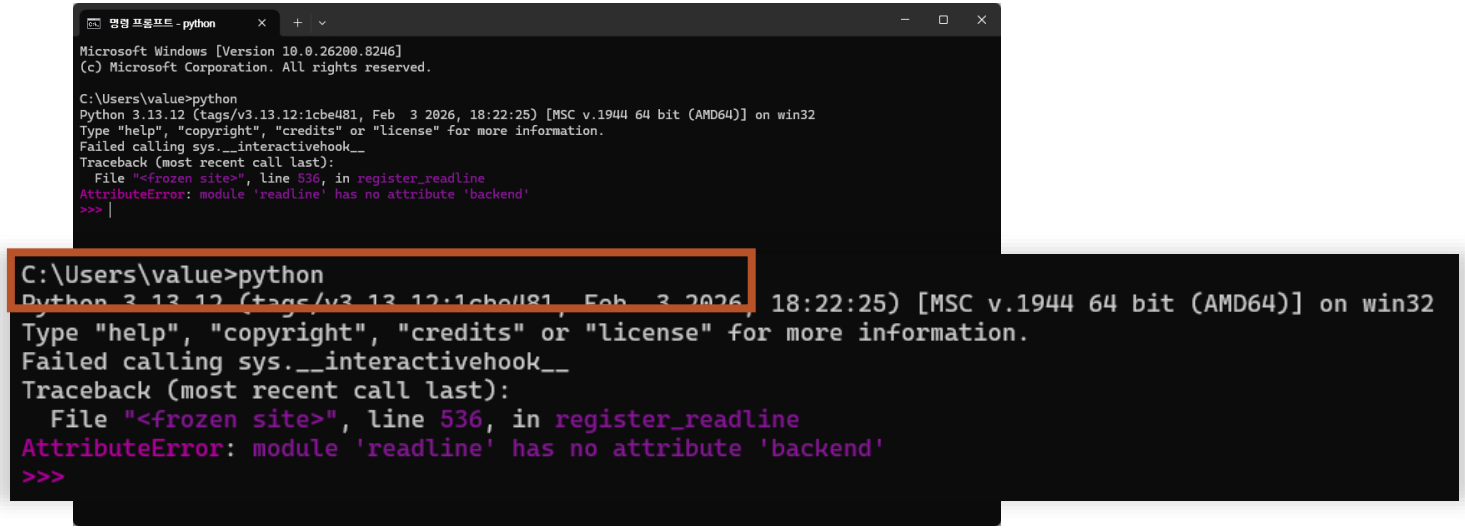
여기서 한 줄씩 코드를 입력하고 즉시 결과를 확인할 수 있습니다.

종료는 exit() 입력.

방법2:

메모장이나 VS Code에서 .py 파일을 작성한 뒤, 명령 프롬프트에서 해당 파일을 실행

python my_analysis.py



M2. 코딩에이전트로 Python 업무자동화

실습 1 — PDF에서 텍스트·표 추출해 엑셀로 옮기기

[워크플로우 개요]

부서에 쌓여 있는 PDF — 민원 접수, 회의록, 통계 보고서, 정책 자료 — 는 그 자체로는 검색·집계가 어렵다. 본 실습은 LLM에게 「이 PDF에서 이런 항목을 뽑아 엑셀로 정리해 달라」고 요청해 받은 코드를 실행하는 가장 기본적인 자동화다. 한 번 만들어두면 양식이 비슷한 PDF가 매달·매주 들어와도 같은 코드로 반복 처리할 수 있다. 「PDF는 보는 문서」에서 「PDF는 처리 가능한 데이터원」으로 인식이 바뀌는 첫 경험이다.

#	시나리오	입력 [예시파일]	출력	난이도
A	민원 접수 PDF → 엑셀 정리	민원접수_2025.pdf	민원접수.xlsx (날짜·민원인·내용 컬럼)	★
B	회의록 PDF → 안건별 시트 분리	회의록_2025_3분기.pdf	회의록.xlsx (안건마다 시트 1개)	★★
C	통계 보고서 PDF의 표 → 엑셀 표	2025_재정통계.pdf	재정통계.xlsx (표 1:1 변환)	★★
D	다수 정책 자료 PDF (10개+) 일괄 → 1개 엑셀	정책자료/*.pdf	정책자료_통합.xlsx (파일별 시트)	★★★
E	스캔된 PDF의 텍스트 영역만 → 엑셀	공문_스캔.pdf (텍스트 레이어 있음)	공문.xlsx	★★

- 작업 폴더에 PDF 준비
- 라이브러리 설치 — pdfplumber pandas openpyxl
- LLM에 요구사항 전달하고 코드 받기
- 받은 코드를 실행 환경(.py·Jupyter·Colab)에서 실행
- 결과 확인 후 누락·오류 있으면 LLM에 에러 메시지 그대로 붙여 재요청

M2. 코딩에이전트로 Python 업무자동화

실습 2 — 폴더 내 다수 CSV를 통합 엑셀 보고서로

[워크플로우 개요]

부서에는 매월·매주·매일 쌓이는 CSV 파일이 많다. 매출, 출장비, 민원 접수, 부서별 통계 — 형식은 비슷하지만 따로 흩어져 있어 통합 보고서를 만들 때마다 사람 손으로 합친다. 본 실습은 LLM에게 「이 폴더 안의 CSV들을 한 엑셀 파일로 통합하고 요약 시트까지 만들어 달라」고 요청하는 패턴이다. 한 번 만들어둔 코드는 다음 달, 다음 분기에도 그대로 다시 쓸 수 있어 부서 정기 보고서 자동화의 출발점이 된다.

#	시나리오	입력 [예시파일]	출력	난이도
A	월별 매출 CSV 12개 → 연간 통합 보고서	매출_2025_01.csv ~ 매출_2025_12.csv	엑셀 1개 (월별 시트 + 연간 요약 시트)	★
B	부서별 출장비 CSV → 부서별 합계	부서마다 CSV 1개	엑셀 1개 (부서명·합계·평균·건수)	★
C	일자별 민원 접수 CSV → 주별·월별 집계	일자 CSV 30~60개	엑셀 1개 (주별·월별 피벗 시트)	★★
D	다수 부서의 통계 CSV (스키마 다름) → 표준 양식 통합	형식이 다른 CSV 여러 개	엑셀 1개 (표준화 후 통합 시트)	★★★
E	CSV 통합 + 차트 자동 삽입	시계열 CSV 다수	엑셀 1개 (데이터 시트 + 차트 시트)	★★

- 작업 폴더에 CSV 파일들 준비 (한 폴더에 모아두기)
- 라이브러리 설치 — pandas openpyxl (차트용은 xlsxwriter 추가)
- LLM에 요구사항 전달 — 입력 폴더 경로·통합 규칙·출력 시트 구성 명시
- 받은 코드 실행
- 결과 엑셀에서 시트 구성·합계·정렬 확인 후 보정 요청

M2. 코딩에이전트로 Python 업무자동화

실습 3 — 워드 양식 일괄 생성

[워크플로우 개요]

부서에는 같은 양식에 사람·날짜·내용만 바뀌어 들어가는 워드 문서가 많다. 표창장, 임명장, 안내 공문, 회의 통지, 인증서, 발송 라벨 — 한 번에 수십·수백 건을 만들어야 할 때 사람 손으로 하면 반나절이 사라진다. 본 실습은 엑셀 표 한 장(수신자 명단)과 워드 템플릿 한 장(자리표시자가 있는 양식)을 LLM에게 주고, 둘을 결합해 개별 워드 파일들을 일괄 생성하는 코드를 받는 패턴이다. 워드 「우편물 머지」 기능의 파이썬 버전인데, 양식 변형·조건 분기·PDF 일괄 변환까지 자유롭게 확장할 수 있다는 차이가 있다.

#	시나리오	입력 [예시파일]	출력	난이도
A	표창장·인증서 일괄 생성	명단 엑셀 + 표창장 워드 템플릿	수상자별 워드 파일 N개	★
B	회의 통지 공문 일괄 발송용 생성	참석자 엑셀 + 공문 템플릿	참석자별 워드 파일 N개	★
C	부서별 맞춤 안내 공문 (조건 분기)	부서 엑셀 + 템플릿	부서별 다른 문구가 들어간 워드 파일 N개	★★
D	워드 일괄 생성 + PDF 변환	명단 + 템플릿	워드 N개 + PDF N개	★★
E	일괄 생성 + 파일명 자동 규칙 + 폴더 분류	명단 + 템플릿	부서·날짜별 폴더에 정리된 워드 파일 N개	★★★

- 워드 템플릿 작성 — 바뀌는 부분에 자리표시자 표기 (예: {{이름}}, {{날짜}}, {{소속}})
- 명단 엑셀 준비 — 자리표시자 이름과 같은 컬럼명으로 정리
- 라이브러리 설치 — python-docx pandas openpyxl (PDF 변환은 docx2pdf 추가)
- LLM에 요구사항 전달 — 템플릿 경로·명단 경로·자리표시자 매핑·파일명 규칙 명시
- 받은 코드 실행 후 출력 폴더의 결과 워드 파일 무작위 3~5건 열어 확인

M2. 코딩에이전트로 Python 업무자동화

실습 4 — Python으로 HWPX(한글) 일괄 생성

[워크플로우 개요]

공공 부문에서 가장 많이 쓰는 문서 형식은 워드가 아니라 한글(HWP/HWPX)이다. 그런데 한글 자동화는 워드보다 진입 장벽이 높았다. 다행히 최근 몇 년 사이 GitHub에 HWPX를 다루는 오픈소스 라이브러리들이 정착했고, Claude Skill·MCP 형태로 「행정안전부 가이드라인을 따르는 HWPX 생성 패키지」도 공유되고 있다. 본 실습은 남이 만들어둔 Python 라이브러리·스킬을 빌려와서 한글 문서를 일괄 생성하는 패턴이다. 핵심은 「코드를 처음부터 짜지 않는다 — 검색하고, 받아오고, 끼워 맞춘다」다. 이 감각을 익히면 부서 문서 90%를 자동화할 수 있다.

#	시나리오	입력 [예시파일]	출력	난이도
A	명단 엑셀 → 임명장·표창장 HWPX 일괄 생성	명단 엑셀 + HWPX 템플릿	개인별 HWPX N개	★
B	회의 안건 정리표 → 회의록 HWPX 자동 작성	안건 엑셀 또는 회의록 텍스트	행안부 양식 회의록 HWPX 1개	★★
C	데이터 분석 결과 → 보고서 HWPX 자동 생성	분석 결과 텍스트·표	표지·목차·본문 갖춘 보고서 HWPX 1개	★★
D	부서별 통계 → 부서별 맞춤 보고서 HWPX	부서 통계 엑셀 + 템플릿	부서마다 다른 내용 HWPX N개	★★★
E	마크다운 문서 → 행안부 양식 HWPX 변환	.md 파일	가이드라인 따르는 HWPX 1개	★★

- GitHub 또는 Skill 저장소에서 사용할 라이브러리·스킬 선정 (강사 사전 준비)
- 라이브러리 설치 또는 스킬 폴더 배치
- 한글 템플릿 준비 — 자리표시자 표기한 HWPX 1개 (시나리오 A·D) 또는 가이드라인 템플릿 활용 (시나리오 B·C·E)
- LLM에 요구사항 전달 — 사용할 라이브러리·스킬 이름 명시, 입력·출력·자리표시자 매핑 설명
- 받은 코드 실행 후 결과 HWPX를 한글에서 직접 열어 서식·내용 확인

M3. HTML/CSS/JS 단일파일 웹앱

M3. HTML/CSS/JS 단일파일 웹앱

단일파일 웹앱이란 무엇인가

[워크플로우 개요]

웹사이트는 보통 여러 개의 파일과 서버를 필요로 한다고 알려져 있지만, 사실 HTML 파일 하나만으로도 작동하는 웹앱을 만들 수 있다. 한 파일 안에 화면 구조(HTML)·디자인(CSS)·동작(JavaScript)을 모두 담고, 라이브러리는 인터넷에 있는 CDN 링크로 불러오면 끝이다. 더블클릭만 하면 브라우저에서 바로 열린다. 서버 설치, 백엔드 구성, 데이터베이스 연결이 전혀 필요 없다. 이 가벼움이 폐쇄망 부서에서 큰 의미를 갖는다 — 동료에게 파일 하나만 보내면 그 사람도 똑같이 도구를 쓸 수 있다. 본 모듈은 「LLM에 단일 HTML 코드 한 덩어리를 받아 더블클릭으로 실행하는」 가장 간단한 웹 자동화 패턴을 다룬다.

영역	역할	위치
<head>	메타 정보·외부 라이브러리 링크·CSS 스타일	파일 상단
<body>	화면에 보이는 구조 (HTML 요소들)	가운데
<script>	사용자 동작 처리·계산·데이터 흐름 (JavaScript)	보통 하단

직접 라이브러리 파일을 다운받아 넣지 않고, 인터넷 상의 주소(CDN)를 <head>에 적기만 하면 된다. 브라우저가 그 주소에서 라이브러리를 받아온다.

- .html 확장자로 저장 (예: 예산계산기.html)
- 더블클릭하면 기본 브라우저에서 열림
- 수정이 필요하면 메모장·VS Code 등으로 열어 편집
- 동료에게 전달할 때는 그 파일 하나만 보내면 끝

폐쇄망 주의점: CDN은 인터넷 연결이 있어야 작동. 완전 폐쇄망에서는 라이브러리 파일을 다운받아 같은 폴더에 넣고 상대 경로로 부름. LLM에 "CDN 대신 인라인 임베드 또는 같은 폴더 파일 참조" 요청.

가능한 것	어려운 것
사용자 화면·폼·버튼·표·차트	여러 사용자가 데이터 공유
입력·계산·검증·결과 표시	회원가입·로그인
브라우저에 데이터 저장 (다음 모듈 M4)	서버에서 데이터 처리
파일 업로드·다운로드 (브라우저 안에서)	외부 시스템과 자동 연동
외부 공개 API 호출	부서 내부 DB와 연결
인쇄·PDF 저장	백그라운드 자동 실행

M3. HTML/CSS/JS 단일파일 웹앱

실습 1 — 부서 안내 페이지·소개 사이트

[워크플로우 개요]

부서 안내·사업 소개·행사 페이지처럼 「보여주는 용도」의 정적 웹페이지는 단일 HTML 파일로 충분하다. 디자인 좋은 안내 페이지를 만들어 부서 내부 공유 폴더에 두거나, 동료에게 파일 하나만 보내도 그대로 열린다. 본 실습은 LLM에게 부서 정보·구조·연락처 등을 자연어로 설명하고 한 번에 완성된 HTML 파일을 받아 보는 가장 단순한 웹 자동화다. 「내가 손으로 워드 안내문을 만들던 것」을 「LLM이 만든 웹 페이지로 대체하는」 첫 경험이 된다. 이미지·연락처·지도 같은 요소를 추가해 가면서 단일파일의 가능 범위를 체감한다.

#	시나리오	입력 [예시파일]	출력	난이도
A	부서 소개 페이지 (조직도·연락처·업무)	부서 정보 자연어 설명	부서소개.html 1개	★
B	사업 안내 페이지 (개요·일정·신청 방법)	사업 정보 + 일정표	사업안내.html 1개	★
C	행사 안내 페이지 (지도·연락처·자주 묻는 질문)	행사 정보 + 장소 좌표	행사안내.html 1개 (Leaflet 지도 포함)	★★
D	정책 자료 정리 페이지 (자료 목록·다운로드 링크)	자료 목록 표	정책자료.html 1개 (검색·필터 기능)	★★
E	인포그래픽 형식 부서 현황 페이지	통계 데이터	부서현황.html 1개 (Chart.js 차트 포함)	★★

- 페이지에 들어갈 정보 정리 (텍스트·이미지·연락처 등)
- LLM에 요구사항 전달 — 페이지 구조·디자인 톤·사용 라이브러리 명시
- 받은 코드를 메모장에 붙여 .html로 저장
- 더블클릭으로 브라우저에서 열기
- 부분 수정 요청 (색감·배치·내용 변경) → 다시 받기 반복
- 만족스러우면 부서 내부 공유 또는 정적 호스팅 (M11)으로 배포

M3. HTML/CSS/JS 단일파일 웹앱

실습 2 — 단일파일 업무 도구 (계산기·체크리스트·폼)

[워크플로우 개요]

부서 업무에는 「엑셀로 만들면 너무 무겁고, 워드는 부적합한」 작은 계산·점검 도구가 많다. 출장비 계산, 점검표 체크리스트, 신청서 양식, 기준 적합성 검토 같은 일들이다. 본 실습은 이런 「작지만 자주 쓰는 도구」를 단일 HTML 파일로 만드는 패턴이다. 사용자가 값을 입력하면 자동으로 결과를 계산해 주거나, 항목을 체크하면 적합 여부를 판정해 주는 식이다. 이전 실습(M3-S2)이 「보여주는 페이지」였다면, 이번 실습은 「상호작용하는 도구」를 만드는 것이다. 실시간 계산·검증·결과 표시가 모두 브라우저 안에서 일어난다. 결과를 인쇄하거나 PDF로 저장하는 것도 가능하다. 부서에 한 번 만들어 두면 동료들이 매일 쓰게 되는 도구가 탄생한다.

#	시나리오	입력 [예시파일]	출력	난이도
A	출장비·여비 계산기	출장 일수·거리·등급	자동 계산된 여비 합계 + 인쇄 가능한 명세서	★
B	부서 점검 체크리스트	항목 체크·비고 입력	점검 결과 요약 + 미흡 항목 강조	★
C	사업 신청서 자동 검증 폼	신청자 정보·금액·기간	입력 즉시 적합성 검증 + 제출 버전 출력	★★
D	보조금 지원 자격 자가진단 도구	신청자 답변 (객관식)	지원 가능 여부 + 추천 사업 제시	★★
E	행정 통계 즉석 계산기 (인구·면적·예산)	수치 입력	비율·증감률·차트 즉시 갱신	★★

- 도구가 처리할 입력·출력·계산 규칙 정리
- LLM에 요구사항 전달 — 화면 구성·계산식·검증 조건·결과 표시 방식 명시
- 받은 HTML 파일을 더블클릭으로 열어 작동 확인
- 입력값 다양하게 넣어 보면서 오작동·누락 발견
- 발견한 문제를 LLM에 다시 알려 보정 요청
- 최종 버전을 부서 공유 폴더에 배치 또는 동료에게 전달

M3. HTML/CSS/JS 단일파일 웹앱

예시 프롬프트

LLM 요청 프롬프트 예시 (실습1)

다음 정보로 부서 소개 단일 HTML 페이지를 만들어줘.
파일 형식: HTML·CSS·JS 모두 한 파일에, Tailwind CSS는 CDN으로
부서명: 데이터정책과인원: 12명 (과장 1, 팀장 3, 주무관 8)
주요 업무: 공공데이터 개방, 데이터 거버넌스, AI 행정 정책 수립
연락처: 전화 02-XXX-XXXX,
이메일 dataX@gov.kr
구조:
상단 헤더 (부서명·표어) /
중앙 조직도 / 하단 업무 영역 카드 3개 / 푸터 연락처톤: 깔끔한 모노톤, 공공기관 분위기,
정렬 깔끔하게

LLM 요청 프롬프트 예시 (실습2)

다음 작업을 수행하는 단일 HTML 파일을 만들어줘.
•파일 형식: HTML·CSS·JS 한 파일, Tailwind CSS는 CDN으로
•목적: 출장비 자동 계산기
•입력: 출장지(시도) / 출장 일수 / 직급(1~5급, 6급 이하) / 교통수단(KTX·고속버스·자가용)
•계산 규칙:
- 일비: 1~5급 25,000원, 6급 이하 20,000원 × 출장 일수
- 식비: 직급별 차등, 일수 곱
- 교통비: 수단별 정해진 단가 적용

•출력: 항목별 금액·합계 표 + 「인쇄」 버튼 (인쇄 시 깔끔한 명세서 형태)
•검증: 음수 입력·빈 값은 경고 표시
•톤: 모노톤, 공공기관 양식 느낌

M3. HTML/CSS/JS 단일파일 웹앱

실습 1 — 메모·할일 앱

[워크플로우 개요]

「내가 오늘 해야 할 일」, 「회의에서 기록한 메모」, 「떠오른 아이디어」 같은 짧은 글을 부서 PC에 따로 저장해 두는 도구는 의외로 마땅한 게 없다. 워드는 무겁고, 메모장은 검색이 안 되고, 포스트잇은 사라진다. 본 실습은 단일 HTML 파일 하나로 「내 PC에서만 작동하는 메모·할일 관리 도구」를 만든다. 입력하면 즉시 저장되고, 브라우저를 닫았다 켜도 그대로 있다. 완료된 항목은 표시하거나 삭제할 수 있고, 검색·정렬도 가능하다. 부서마다 한두 명은 이런 도구를 매일 쓰게 된다 — 회의록 정리 담당자, 민원 추적 담당자처럼. 한 번 만들어두면 본인 업무에 영구히 따라다니는 「개인 도구」가 탄생한다.

#	시나리오	입력 [예시파일]	출력	난이도
A	단순 할일 앱	할일 텍스트	체크·삭제·초기화 가능한 목록	★
B	분류 가능한 할일 앱	할일 + 카테고리 (업무·개인·긴급)	카테고리별 필터링 가능한 목록	★
C	메모장	제목 + 본문 + 작성일	검색·필터 가능한 메모 목록	★★
D	회의 메모 도구	회의명·날짜·참석자 + 메모	회의별로 묶인 메모 묶음	★★
E	우선순위 할일 앱	할일 + 중요도·마감일	자동 정렬된 우선순위 목록	★★★

새로고침해도 사라지지 않는 첫 도구. 단일 HTML이 비로소 「내 PC에서 매일 쓰는 개인 도구」로 격상된다.
부서 누구나 한두 개씩 갖게 되는 작은 자동화의 시작점.

- 도구가 다룰 데이터의 형태 정리 (제목·내용·완료여부·작성일 등)
- LLM에 요구사항 전달 — 화면 구성·저장 시점·불러오기 시점·삭제 방식 명시
- 받은 HTML을 더블클릭으로 열어 작동 확인
- 항목 입력·새로고침·재방문 시나리오로 영속성 검증
- 부족한 기능(검색·정렬·필터) 보완 요청
- 부서 공유 폴더에 배치하거나 본인 PC에 영구 보관

M3. HTML/CSS/JS 단일파일 웹앱

실습 2 — 단일파일 업무 도구 (계산기·체크리스트·폼)

[워크플로우 개요]

부서 업무에는 「엑셀로 만들면 너무 무겁고, 워드는 부적합한」 작은 계산·점검 도구가 많다. 출장비 계산, 점검표 체크리스트, 신청서 양식, 기준 적합성 검토 같은 일들이다. 본 실습은 이런 「작지만 자주 쓰는 도구」를 단일 HTML 파일로 만드는 패턴이다. 사용자가 값을 입력하면 자동으로 결과를 계산해 주거나, 항목을 체크하면 적합 여부를 판정해 주는 식이다. 이전 실습(M3-S2)이 「보여주는 페이지」였다면, 이번 실습은 「상호작용하는 도구」를 만드는 것이다. 실시간 계산·검증·결과 표시가 모두 브라우저 안에서 일어난다. 결과를 인쇄하거나 PDF로 저장하는 것도 가능하다. 부서에 한 번 만들어 두면 동료들이 매일 쓰게 되는 도구가 탄생한다.

#	시나리오	입력 [예시파일]	출력	난이도
A	출장비·여비 계산기	출장 일수·거리·등급	자동 계산된 여비 합계 + 인쇄 가능한 명세서	★
B	부서 점검 체크리스트	항목 체크·비고 입력	점검 결과 요약 + 미흡 항목 강조	★
C	사업 신청서 자동 검증 폼	신청자 정보·금액·기간	입력 즉시 적합성 검증 + 제출 버전 출력	★★
D	보조금 지원 자격 자가진단 도구	신청자 답변 (객관식)	지원 가능 여부 + 추천 사업 제시	★★
E	행정 통계 즉석 계산기 (인구·면적·예산)	수치 입력	비율·증감률·차트 즉시 갱신	★★

부서 업무의 빈틈을 메우는 작은 도구는 단일 HTML 한 장이면 충분하다. 출장비 계산, 점검 체크리스트, 신청서 검증 — 매일 손으로 하던 일이 더블클릭 한 번으로 끝난다. **파일 하나가 부서 도구가 된다.**

- 도구가 처리할 입력·출력·계산 규칙 정리
- LLM에 요구사항 전달 — 화면 구성·계산식·검증 조건·결과 표시 방식 명시
- 받은 HTML 파일을 더블클릭으로 열어 작동 확인
- 입력값 다양하게 넣어 보면서 오작동·누락 발견
- 발견한 문제를 LLM에 다시 알려 보정 요청
- 최종 버전을 부서 공유 폴더에 배치 또는 동료에게 전달

M4. 클라이언트 영속성 (localStorage · IndexedDB)

M4. 클라이언트 영속성 (localStorage · IndexedDB)

브라우저가 데이터를 기억한다

[워크플로우 개요]

앞 모듈에서 만든 단일 HTML 도구의 한계는 분명했다. 새로고침하면 입력했던 값이 사라진다. 브라우저를 닫았다가 다시 열면 처음 상태로 돌아간다.
「데이터를 기억하지 못하는 도구」였다. 본 모듈은 이 한계를 서버 없이 해결하는 방법을 다룬다. 브라우저 자체에 작은 저장 공간이 있다.
사용자의 PC, 그 브라우저 안에만 데이터가 머무른다. 서버도 데이터베이스도 필요 없다. 새로고침해도, 브라우저를 껐다 켜도 데이터가 그대로 있다.
다음 날 같은 PC에서 같은 파일을 열면 어제 입력한 내용이 그대로 남아 있다. 이 가능성이 「단순한 단일파일」을 「개인 업무 도구」로 격상시킨다.

브라우저 저장 공간 — 두 가지 방식

브라우저에 데이터를 저장하는 표준적인 방법은 두 가지다. localStorage와 IndexedDB.

localStorage — 가벼운 키-값 저장소

가장 단순한 방식. 「이름표」와 「내용」 한 쌍을 저장한다. 예를 들어 회원이름 = "김철수" 처럼. 텍스트만 저장하고, 용량 제한도 적당히 있다(보통 5~10MB). 코드 한두 줄로 저장·불러오기가 끝난다. 부서 도구의 90%는 이걸로 충분하다.

IndexedDB — 구조화된 데이터 저장소

더 큰 데이터, 복잡한 구조의 데이터(여러 항목·검색·필터)가 필요할 때 쓴다. 작은 데이터베이스가 브라우저 안에 들어 있다고 보면 된다. 코드는 약간 복잡하지만 LLM이 다 짜준다. 본 모듈에서는 「큰 데이터가 필요한 시나리오에서만 IndexedDB」 정도로 알고 있으면 된다.

- **브라우저는 이 데이터를 사용자 PC의 특정 폴더에 자동 저장한다.** 사용자가 따로 「저장 버튼」을 누를 필요가 없다. JavaScript 코드 한 줄로 저장하면 끝. 다음에 같은 페이지를 열면 그 데이터를 읽어와서 화면에 다시 보여준다.
- **이 데이터는 브라우저별·PC별로 독립적이다.** 같은 사람이 회사 PC에서 저장한 데이터는 집 PC에서는 안 보인다. 동일 PC라도 크롬에서 저장한 것은 엣지에서 안 보인다. 「개인용 도구」로는 적합하지만 「여럿이 공유하는 도구」에는 부적합하다.

M4. 클라이언트 영속성 (localStorage · IndexedDB)

가능 범위와 한계

서버 없이도 가능한 일이 늘어난다. 메모, 할일 목록, 자주 쓰는 항목 즐겨찾기, 입력하다 만 폼 임시저장, 사용자 설정 기억 — 이런 것들이 모두 단일 HTML 파일 안에서 작동한다.
반면 한계도 분명하다. 같은 데이터를 여러 사람이 같이 쓸 수는 없다. 동료에게 파일을 보내도 데이터는 따라가지 않는다(빈 도구만 전달된다).
백업·복원도 직접 만들어야 한다. 브라우저 캐시를 청소하면 데이터가 사라진다.

「내가 매일 혼자 쓰는 도구」 → localStorage로 충분.
「부서 동료들과 공유하는 도구」 → M7 이후 클라우드 풀스택으로.

LLM에 영속성 기능을 잘 받기 위한 4가지

- 저장할 데이터의 형태 명시 — 「할일 목록 (제목·완료여부·작성일)」
- 언제 저장할지 명시 — 「입력 즉시 자동 저장」 또는 「버튼 클릭 시 저장」
- 언제 불러올지 명시 — 「페이지 열 때 자동으로 화면에 표시」
- 삭제 방식 명시 — 「개별 삭제 가능, 전체 초기화 버튼」

단일 HTML 파일이 「데이터를 기억하는 개인 업무 도구」가 된다. 새로고침해도, 닫았다가 열어도 그대로다. 서버 없이, 데이터베이스 없이, 파일 하나로. 「내가 매일 쓰는 작은 도구」를 만들 수 있는 단계로 올라선다.

M4. 클라이언트 영속성 (localStorage · IndexedDB)

실습 1 — 데이터 백업·복원 가능한 개인 기록 도구

[워크플로우 개요]

브라우저에 저장한 데이터는 「내 PC, 내 브라우저」 안에만 있다. 사용자에겐 편하지만 한계도 분명하다. 다른 PC로 옮길 수 없고, 브라우저 캐시가 청소되면 사라지고, 동료에게 데이터를 전달할 방법이 없다. 본 실습은 이 한계를 해결하는 핵심 기능 — 「JSON·CSV로 내보내기 / 다시 불러오기」 를 갖춘 개인 기록 도구를 만든다. 사용자가 「내보내기」 버튼을 누르면 모든 데이터가 한 파일로 떨어지고, 그 파일을 다른 PC에서 「불러오기」하면 그대로 복원된다. 이 기능이 단일파일 도구의 활용 폭을 결정적으로 넓힌다 — 회사 PC에서 작성한 기록을 노트북으로 옮기고, 동료에게 양식만이 아니라 「실제 데이터까지 포함된 도구」를 전달할 수 있다. 「내 데이터는 내가 통제한다」는 감각이 자리잡는 실습이다.

#	시나리오	입력 [예시파일]	출력	난이도
A	일일 업무 일지	날짜·업무내용·시간·비고	JSON 내보내기·불러오기	★
B	민원 처리 이력 추적	민원번호·접수일·처리일·상태·내용	JSON·CSV 둘 다	★★
C	부서 학습 노트 (책·강의·세미나)	제목·일자·요점·평가	JSON·CSV 둘 다	★★
D	이슈 트래커 (개선 과제 추적)	이슈·우선순위·상태·담당·이력	JSON 내보내기 + CSV 보고용	★★
E	주간 회고·일기	날짜·잘한 일·아쉬운 일·다음 주 계획	JSON·암호화 옵션	★★★

「내 PC 안에만 머무르는 데이터」를 「내가 통제하는 데이터」로 격상시킨다. 내보내기·불러오기 버튼 두 개가 단일파일 도구의 활용 폭을 결정적으로 넓힌다. 회사 PC에서 노트북으로, 동료에게서 동료에게, 작년에서 올해로 — 데이터가 이동하기 시작한다. **「작은 도구가 부서 자산이 되는」 첫 분기점.**

- 누적할 데이터 항목 정리 (도구마다 4~6개 필드)
- LLM에 요구사항 전달 — 입력·표시·검색·내보내기·불러오기 모두 명시
- 받은 HTML을 더블클릭으로 열어 작동 확인
- 데이터 5~10건 입력 후 「내보내기」 → 다운받은 파일 확인
- 브라우저 데이터 모두 지우고 「불러오기」로 복원 검증
- 다른 PC에 파일 가져가서 불러오기 → 그쪽에서도 동일하게 작동하는지 확인

M4. 클라이언트 영속성 (localStorage · IndexedDB)

실습 2 — 즐겨찾기·북마크 관리 도구

[워크플로우 개요]

부서 업무에는 자주 들어가는 사이트·자료·시스템이 많다. 공공데이터 포털, 법제처, 통계청, 부처 내부 시스템, 자주 쓰는 양식 다운로드 페이지 등이다. 브라우저 즐겨찾기에 다 넣으면 금세 어지러워지고, 분류·검색이 어렵다. 본 실습은 단일 HTML 파일 하나로 **「부서 맞춤 즐겨찾기 도구」**를 만든다. 사이트 이름·URL·분류·메모를 함께 저장하고, 분류별로 묶거나 키워드로 검색할 수 있다. 한 번 만들어 두면 부서 신입에게 파일 하나만 건네줘도 「우리 부서가 자주 쓰는 사이트」가 그대로 전달된다(빈 도구만 가지만, 본인이 채워 가면 본인 업무 지도가 된다). 메모·할일이 「개인 기록 도구」였다면, 즐겨찾기는 「개인 업무 지도」다.

#	시나리오	입력 [예시파일]	출력	난이도
A	단순 즐겨찾기	사이트명 + URL	클릭 가능한 링크 목록	★
B	분류 가능한 즐겨찾기	사이트명 + URL + 카테고리	카테고리별 묶인 링크 목록	★
C	검색·메모 가능한 즐겨찾기	사이트명 + URL + 메모 + 태그	키워드 검색·태그 필터 가능한 목록	★★
D	부서 자료 라이브러리	자료명 + 위치(URL 또는 파일경로) + 설명	분류·검색 가능한 자료 목록	★★
E	자주 쓰는 양식·서식 모음	양식명 + 다운로드 링크 + 사용 방법	카테고리별 양식 패널 + 빠른 다운로드	★★★

개인 업무 지도가 단일 파일 안에 만들어진다. 자주 가는 사이트·자료·양식이 한 곳에 모이고 분류·검색이 가능해진다. 부서 동료에게 빈 도구를 전달하면 그쪽에서도 본인 지도를 그릴 수 있다. **「작은 도구의 누적」이 부서의 작은 인프라가 된다.**

- 다룰 데이터 항목 정리 (사이트명·URL·카테고리·메모 등)
- LLM에 요구사항 전달 — 화면 구성·추가 방식·검색 방식·정렬 방식 명시
- 받은 HTML을 더블클릭으로 열어 작동 확인
- 사이트 5~10건 추가해서 분류·검색·삭제 검증
- 새로고침·재방문 시 데이터 유지 확인
- 본인 부서 자주 쓰는 사이트 10~20건으로 채워 「본인용 업무 지도」 완성

M4. 클라이언트 영속성 (localStorage · IndexedDB)

실습 3 — 폼 입력 임시저장 기능

[워크플로우 개요]

부서에서 자주 쓰는 신청서·결재 양식·점검표는 한 번에 다 채우기 어려운 경우가 많다. 중간에 자료를 찾아야 하거나, 다른 일이 끼어들거나, 상사 결재 후 보완해야 한다. 이때 브라우저를 닫거나 새로고침하면 입력했던 내용이 모두 사라진다. 사용자에게는 작지만 매번 반복되는 짜증의 원인이다. 본 실습은 단일 HTML 양식에 「입력하는 동안 자동으로 임시저장되고, 다음에 다시 열어도 그대로 남아 있는」 기능을 붙인다. 사용자가 따로 「저장」 버튼을 누르지 않아도 글자를 입력할 때마다 백그라운드에서 저장된다. 작성을 마치고 「제출」 버튼을 누르면 인쇄·PDF·이메일 발송용 최종본이 생성되고 임시저장은 비워진다. 이 패턴은 부서의 모든 양식 도구에 적용 가능한 보편 패턴이라 한 번 익히면 응용 폭이 크다.

#	시나리오	입력 [예시파일]	출력	난이도
A	출장 신청서 임시저장 폼	출장지·일정·목적·예산	임시저장 + 최종 인쇄용 신청서	★
B	회의록 작성 폼	회의명·일시·참석자·안건·결정사항	자동 저장되는 회의록 + 최종 출력본	★
C	점검 체크리스트 폼	점검 항목 체크 + 비고	진행 중 자동 저장 + 점검 결과 보고서	★★
D	사업 신청서 폼 (긴 양식)	사업명·예산·기간·인력·기대효과	섹션별 자동 저장 + 최종 PDF 출력	★★
E	다단계 폼 (위저드)	단계별 입력 (1단계→2단계→3단계)	각 단계 진행 상황 자동 저장 + 종합 결과	★★★

사용자가 「저장」 버튼을 누르지 않아도 자동으로 저장된다. 폼 입력 중간에 자리를 떠도, 브라우저가 꺼져도, 다음 날 다시 열면 그대로 있다. 이 작은 배려가 양식 도구의 사용성을 결정한다. **「자동 저장은 사치가 아니라 기본」**이라는 감각이 부서 도구 품질을 한 단계 끌어올린다.

- 양식의 입력 항목 목록 정리 (텍스트·날짜·선택·체크 등)
- LLM에 요구사항 전달 — 항목 구성·자동 저장 시점·제출 동작·최종 출력 형태 명시
- 받은 HTML을 더블클릭으로 열어 작동 확인
- 일부 입력 후 새로고침·브라우저 종료 → 재방문 시 입력값 복원 확인
- 「제출」 후 임시저장 비워짐 확인
- 부서에서 실제 쓰는 양식 1건으로 응용

M4. 클라이언트 영속성 (localStorage · IndexedDB)

localStorage 보관 기간의 실제

사라지는 조건	빈도·위험도
사용자가 브라우저 「캐시·쿠키 삭제」 또는 「인터넷 사용 기록 삭제」 실행	높음 (가장 흔함)
시크릿 모드·InPrivate 창에서 작성	매우 높음 (창 닫는 순간 사라짐)
브라우저 자체 자동 정리 정책 (Safari iOS는 7일 미사용 시 삭제)	브라우저 의존
브라우저별·도메인별 용량 한도 초과 (보통 5~10MB)	데이터 누적 시 발생
다른 브라우저에서 같은 페이지 열기	항상 (브라우저 간 격리)
다른 PC에서 같은 페이지 열기	항상 (PC 간 격리)
윈도우 사용자 계정 삭제·OS 재설치	매우 높음 (모두 소실)
HTML 파일 위치 변경 (file:// 경로 바뀜)	중간 (브라우저별로 다름)

•file:// 프로토콜의 한계HTML 파일을 더블클릭으로 열면 file:///C:/... 경로로 작동한다. 이때 localStorage는 파일 경로별로 격리되거나, 일부 브라우저에서는 저장이 아예 안 되거나, 여러 HTML이 같은 저장소를 공유해 충돌하는 등 일관되지 않게 작동한다. Chrome은 비교적 안정적이지만 Safari·Firefox는 차이 있음.

•localStorage는 「영구 저장」이 아니라 **「당분간 보관」**으로 이해해야 한다
•중요한 데이터는 반드시 정기적으로 JSON 내보내기로 백업
•부서 공식 기록·법정 보관 의무 데이터는 localStorage에 두면 안 된다 — 클라우드(M7~) 또는 부서 공식 시스템으로
•localStorage는 「개인 작업 메모·임시 양식·자주 쓰는 즐겨찾기」 정도가 적정선

M4. 클라이언트 영속성 (localStorage · IndexedDB)

「저장」이 아니라 「읽어서 메모리에 띄우기」

[워크플로우 개요]

CSV 시각화 도구는 사용자가 파일을 선택하면 브라우저가 그 파일을 메모리(RAM)에 읽어 들여 처리하는 구조다. 서버에 올리지도 않고, localStorage에 저장하지도 않는다. 화면을 새로고침하거나 페이지를 닫으면 그 데이터는 사라진다. 다음에 같은 시각화를 보려면 같은 파일을 다시 선택해야 한다

사라지는 조건	빈도·위험도
1. 사용자가 「파일 선택」 버튼으로 CSV 지정	브라우저 파일 선택창
2. JavaScript의 FileReader가 파일 내용을 읽음	브라우저 메모리
3. CSV 텍스트를 행·열 데이터 객체로 변환 (Papa Parse 라이브러리 등)	브라우저 메모리
4. 차트 라이브러리(Chart.js·D3)에 데이터 전달	브라우저 메모리
5. 화면에 차트·표 렌더링	브라우저
6. 페이지 닫으면 데이터 소멸	—

- 핵심: 파일은 사용자 PC를 떠나지 않는다. 외부에 전송되지 않고, 어디에도 저장되지 않는다. 폐쇄망 부서에서 외부 데이터 분석 서비스(예: Tableau Public, Google Sheets)를 못 쓸 때 가장 안전한 대안이 이 패턴이다.

- localStorage(5~10MB)와 달리 이 패턴은 저장이 아니라 읽기·처리라서 용량 제약이 다르다.
- 1MB 미만 (수천 행 이하) → 즉각 처리, 차트도 부드러움
- 1~10MB (수만 행) → 약간 지연 있지만 여전히 작동
- 10~100MB (수십만 행 이상) → 처리 가능하나 차트 갱신이 느림, 페이징 필요
- 100MB 이상 → IndexedDB에 청크 단위 적재 또는 서버 쪽 처리 권장

M5. 엑셀 VBA × LLM 어시스턴트

M5. 엑셀 VBA × LLM 어시스턴트

VBA가 부서 PC에서 갖는 힘

[워크플로우 개요]

부서 PC에는 이미 강력한 자동화 도구가 깔려 있다. **엑셀과 함께 들어 있는 VBA(Visual Basic for Applications)**다. 별도 설치도 없고, 인터넷도 필요 없고, 보안 검토도 까다롭지 않다. 그동안 VBA는 「문법을 배워야 쓸 수 있는 도구」로 여겨져 진입 장벽이 높았다. LLM 시대에 이 장벽이 사라졌다. 본 모듈은 「LLM에게 VBA 매크로 코드를 받아 엑셀에 붙여넣고 실행하는」 가장 실용적인 부서 자동화 패턴을 다룬다. Python 자동화(M2)가 「외부에서 받은 코드를 실행 환경에 옮겨 돌리는」 방식이라면, VBA는 「엑셀·아웃룩 안에서 바로 작동하는」 방식이다. 부서 환경이 까다로울수록 VBA의 가치는 높아진다.

VBA가 부서 환경에서 갖는 강점

- 추가 설치 불필요 — Office가 깔린 PC면 즉시 사용 가능
- 외부망 접근 불필요 — 코드는 외부에서 받지만, 실행은 엑셀 안
- 보안 검토 단순 — 매크로 활성화 외에 별도 절차 없음
- 레거시 양식과 직접 결합 — 기존 엑셀 양식·아웃룩 메일을 그대로 자동화
- 부서원 누구나 실행 가능 — Python 환경 없이도 매크로 활성화만으로 작동

- **부서 PC에 이미 들어 있는 가장 강력한 자동화 도구.별도 설치 없이, 외부망 없이, 보안 절차 없이 작동한다. LLM이 문법 장벽을 없앤 지금, VBA는 폐쇄망 부서의 가장 현실적인 자동화 수단이다. 「엑셀 창 안에서 사람이 손으로 하던 일」을 그대로 매크로화하는 것이 본 모듈의 목표다.**

VBA로 다룰 수 있는 범위

- 엑셀 — 셀·시트·차트·피벗·서식·수식 자동화
- 아웃룩 — 메일 자동 분류·자동 회신·일정 관리
- 워드 — 양식 일괄 생성·문서 변환·서식 적용
- 파워포인트 — 데이터 → 슬라이드 자동 생성
- 파일 시스템 — 폴더 탐색·파일 이동·이름 변경
- 다른 엑셀 파일 읽기 — 부서 공유 폴더의 다수 엑셀을 통합
- HTTP 호출 — (제한적이지만) 공공 API 호출 가능

M5. 엑셀 VBA × LLM 어시스턴트

실습 1 — 일일·주간 보고서 자동 생성 매크로

[워크플로우 개요]

부서 매일·매주 반복되는 보고서가 있다. 일일 매출, 주간 민원 처리 현황, 부서별 통계 — 원본 데이터는 매번 새로 들어오지만 보고서 구조는 늘 똑같다. 사람이 매번 같은 손작업을 반복하고, 엑셀 함수로 만들기엔 너무 복잡하다. 본 실습은 **「원본 엑셀의 구조를 LLM에 알려주고, 거기서 자동으로 보고서를 만들어 주는 VBA 매크로」**를 받는 패턴이다. 핵심은 「LLM에 엑셀 전체를 넘기지 않는다」는 점이다. 부서 데이터는 외부에 보내지 않는다. 대신 시트 구조·컬럼명·샘플 5~10행 정도만 보여주고, 그것을 기억한 LLM이 VBA 코드를 만들어 준다. 실제 데이터 처리는 부서 PC의 엑셀 안에서만 일어난다. 한 번 만든 매크로는 매일·매주 같은 구조의 데이터에 그대로 재사용 가능하다.

#	시나리오	입력 [예시파일]	출력	난이도
A	일일 매출 → 일일 보고서	일자·품목·수량·금액·담당자	일자 요약 + 부서 합계 + 차트	★
B	주간 민원 → 주간 처리 현황	민원번호·접수일·처리일·상태·부서	주간 처리율 + 부서별 비교 표	★★
C	일일 출석·근태 → 부서장 보고용 요약	직원·출근시간·외근·연차	부서별 출근 현황 한 페이지 요약	★★
D	월별 예산 집행 → 집행률 보고서	예산항목·계획·실집행·잔액	항목별 집행률 + 경고 색상 표시	★★
E	다수 부서 일일 보고서 통합 → 본부 종합본	부서별 시트 또는 별도 파일 다수	본부 단위 통합 보고서 + 부서별 차트	★★★

「엑셀 구조만 알려주고, 데이터는 안 준다」이 한 줄이 본 실습의 보안 철학이자 운영 철학이다. LLM은 구조로 매크로를 짜고, 부서 PC는 그 매크로로 실제 데이터를 처리한다. 외부 지능과 내부 데이터의 안전한 협업 — 폐쇄망 부서가 외부 LLM을 가장 깊이 활용할 수 있는 방식이다.

- 1단계 — 원본 엑셀의 구조 파악: 시트명·컬럼명·데이터 타입·행 수 정도를 정리
- 2단계 — 일부 데이터만 LLM에 보여주기: 헤더 + 샘플 5~10행을 복사해 LLM에 붙여넣음 (실제 데이터 안 줘도 됨, 가짜 샘플로 충분)
- 3단계 — LLM이 구조를 「기억」하게 만들기: 「이 구조의 엑셀이 있다고 가정」 명시
- 4단계 — 보고서 요구사항 전달: 어떤 시트에 어떤 형태의 결과를 만들지 자연어로 설명
- 5단계 — VBA 코드 받아 엑셀에 붙여넣기: Alt+F11 → 모듈 → 붙여넣기
- 6단계 — 실행·검증: 매크로 실행 후 결과 시트 확인, 오류 시 에러 메시지를 LLM에 다시 붙여 보정 요청
- 7단계 — .xlsm으로 저장: 매크로 포함 형식으로 보관, 부서 공유

M5. 엑셀 VBA × LLM 어시스턴트

실습 2 — 다수 엑셀 파일 통합·분리 매크로

[워크플로우 개요]

부서장은 산하 팀·기관에 「양식대로 채워서 제출하라」고 지시한다. 메일·공유폴더로 수십 개의 엑셀이 들어온다. 합치는 건 사람이 한다 — 한 개씩 열어 복사하고, 본부 통합 파일에 붙여 넣고, 어느 부서가 안 보냈는지 확인하고, 형식이 다른 파일은 손으로 정리한다. 매번 반나절이 사라진다. 본 실습은 **「공유폴더의 다수 엑셀을 자동으로 합치거나, 반대로 한 파일을 부서별로 자동 분리하는 VBA 매크로」**를 만든다. 한 번 만들어 두면 매월·매분기 같은 작업이 1초로 끝난다. 부서 본부·기획·예산·인사 부서에서 가장 흔히 마주치는 「반복 통합 작업」을 해결하는 도구다.

#	시나리오	입력 [예시파일]	출력	난이도
A	공유폴더의 부서별 엑셀 → 본부 통합 파일	폴더 안 부서별 엑셀 다수 (같은 양식)	통합 파일 1개 (부서명 컬럼 추가)	★
B	본부 통합 파일 → 부서별 엑셀 자동 분리	부서명 컬럼이 있는 통합 엑셀	부서별 엑셀 N개 (각 부서 데이터만)	★
C	양식 다른 부서 파일 통합 (컬럼 매핑)	형식이 조금씩 다른 엑셀	표준화 후 통합된 파일 + 매핑 로그	★★
D	미제출 부서 자동 확인 + 통합	제출자 명단 + 받은 파일들	통합 파일 + 미제출 부서 목록	★★
E	분기·월별 누적 통합	매월 들어오는 동일 양식 엑셀	누적 통합 파일 (월 컬럼 자동 추가) + 월별 시트	★★★

처음부터 실제 폴더(수십 개 파일) 통째로 돌리지 않는다. **테스트용 폴더에 3~5개 파일을 복사해서 매크로 검증**부터 한다. 한 번에 잘못된 매크로가 큰 폴더를 망치면 복구가 어렵다. 작은 폴더에서 결과 검증 → 만족스러우면 실제 폴더에 적용하는 흐름이 안전하다.

- 1단계 — 양식 파악: 부서들이 받아 사용하는 표준 양식의 시트명·컬럼명 정리
- 2단계 — 폴더 구조 정리: 통합할 파일들이 있는 공유폴더 경로, 파일명 규칙(매출_데이터정책과_2025_03.xlsx 등)
- 3단계 — 가짜 샘플로 LLM에 양식 알리기: 헤더 + 가짜 데이터 3~5행
- 4단계 — 통합 규칙 명시: 「부서명 컬럼 자동 추가·동일 헤더 발견 시 처리·빈 파일 처리」 등
- 5단계 — VBA 코드 받아 매크로 통합용 빈 엑셀에 붙여넣기
- 6단계 — 실행·검증: 작은 폴더(테스트용 3~5개 파일)로 검증
- 7단계 — .xlsm으로 저장 후 본 폴더에 적용

M6. Ollama + 레거시 폐쇄망

M6. Ollama + 레거시 폐쇄망

Ollama란 무엇인가

Ollama

Ollama는 **내 PC에서 LLM을 돌리는 도구**다. ChatGPT나 Claude처럼 외부 서버에 접속하는 것이 아니라, 모델 파일을 내 컴퓨터에 다운로드해서 **인터넷 없이** 실행

SLM(Small Language Model)이란: GPT-4나 Claude처럼 수천억 개의 파라미터를 가진 거대 모델이 아니라, **10억~70억 개(1B~7B)** 수준의 작은 모델

- GPU 없이 CPU만으로도 돌릴 수 있다 (일반 사무용 PC에서 실행 가능)
- 응답 속도는 느리다 (GPT-4 대비 5~10배 느림)
- 성능은 떨어진다 (복잡한 추론은 약하지만, 간단한 질의응답·요약·분류는 가능)

모델	크기	파라미터	일반 사무 PC에서	특징
llama3.2:1b	1.3GB	10억	쾌적	가장 가벼움. 간단한 질의응답에 적합
llama3.2:3b	2.0GB	30억	쾌적	1b보다 똑똑. 요약·분류에 충분
gemma2:2b	1.6GB	20억	쾌적	구글의 경량 모델. 한국어 지원 양호
phi-3:3.8b	2.3GB	38억	무난	마이크로소프트. 추론 능력 우수
qwen2.5:7b	4.7GB	70억	△ 느림	알리바바. 중국어+영어+한국어. 7B 중 최고 수준
mistral:7b	4.1GB	70억	△ 느림	프랑스 Mistral. 영어 성능 우수

- **ollama.com** 접속 → **Download** → 운영체제에 맞는 설치 파일 다운로드.
- 설치 (Next 연타).
- 끝. 명령 프롬프트에서 ollama 치면 사용법이 나온다.
- ollama run llama3.2:1b

작은 모델일수록 프롬프트가 중요하다

교훈: GPT-4 같은 대형 모델은 막연한 지시에도 알아서 맥락을 파악하지만,
1B 모델은 **프롬프트에 명시적으로 지시해야** 한다.
작은 모델일수록 프롬프트 설계가 정교해야 원하는 결과가 나온다.
이것이 "프롬프트 엔지니어링이 왜 중요한가"의 가장 직관적인 체험이다.

M6. Ollama + 레거시 폐쇄망

Ollama란 무엇인가

Ollama의 모델들

모델	회사	파라미터	실효 파라미터	파일 크기 (Q4)	필요 RAM	Ollama 명령어	특징
Gemma 4 E2B	Google	5B	2B (PLE)	~1.5GB	2GB	`ollama run gemma4:e2b`	2026.4 최신. PLE 기술로 CPU/가속기 분리 실행. 모바일/엣지 최적화. Apache 2.0
Gemma 4 E4B	Google	8B	4B (PLE)	~3GB	3GB	`ollama run gemma4:e4b`	2026.4 최신. E2B보다 강력. 노트북 추천 모델. LMArena 1300+ 달성. 추론 모드(Thinking) 지원
Gemma 4 26B-A4B	Google	26B (MoE)	4B 활성	~8GB	16GB+	`ollama run gemma4:26b-a4b`	Gemma 최초 MoE. 4B만 활성화하여 효율적. GPU 필요
Gemma 3n E4B	Google	8B	4B (PLE)	~3GB	3GB	`ollama run gemma3n:e4b`	2025.6 출시. 오디오/비디오 입력 지원 (Gemma 4는 아직 텍스트+이미지만). 온디바이스 특화
Gemma 3 4B	Google	4B	4B	~2.5GB	4GB	`ollama run gemma3:4b`	코딩+문서+설명 올라운더. "하나만 쓴다면 이것" 추천 다수
Qwen 3.5 4B	Alibaba	4B	4B	~2.5GB	4GB	`ollama run qwen3.5:4b`	262K 컨텍스트. Thinking/Non-thinking 모드 전환. 다국어 최강
Qwen 3.5 0.8B	Alibaba	0.8B	0.8B	~0.5GB	1GB	`ollama run qwen3.5:0.8b`	가장 작은 멀티모달 모델. 262K 컨텍스트. 초경량 엣지용
Qwen 3 4B	Alibaba	4B	4B	~2.5GB	4GB	`ollama run qwen3:4b`	Thinking 모드로 추론 품질 향상. 도구 호출(Tool Calling) 지원
Phi-4-mini	Microsoft	3.8B	3.8B	~2.3GB	4GB	`ollama run phi4-mini`	수학·논리·코딩 벤치마크 최강 (MATH, GPQA). 128K 컨텍스트. 추론 특화
Llama 3.2 3B	Meta	3B	3B	~2.0GB	4GB	`ollama run llama3.2:3b`	도구 호출·구조화 출력 강점. 모바일/엣지 최적화. 가장 가벼운 "실용" 모델
Llama 3.2 1B	Meta	1B	1B	~1.3GB	2GB	`ollama run llama3.2:1b`	초경량. 간단한 분류·Q&A에 충분. CPU에서 즉시 응답
SmolLM3 3B	HuggingFace	3B	3B	~1.8GB	3GB	`ollama run smollm3`	128K 컨텍스트. 도구 호출 지원. 완전 오픈소스(학습 데이터·방법론 공개). 3B급 최고 성능
DeepSeek-R1 1.5B	DeepSeek	1.5B	1.5B	~1.1GB	2GB	`ollama run deepseek-r1:1.5b`	추론(Reasoning) 특화 증류 모델. Qwen2.5 기반. 수학·논리에 강점
Mistral 7B	Mistral AI	7B	7B	~4.1GB	8GB	`ollama run mistral`	파인튜닝 최적. Apache 2.0. 7B 기준 가장 성숙한 생태계

M6. Ollama + 레거시 폐쇄망

Ollama 실습 — 하이퍼파라미터 조정 + 프롬프트 최적화

Modelfile로 하이퍼파라미터 조정

Ollama에서 모델의 동작을 세밀하게 제어하려면 **Modelfile**을 만든다.
Modelfile은 모델의 설정값(하이퍼파라미터)과 시스템 프롬프트를 담은 텍스트 파일이다.

```
•FROM gemma4:e4b

•# 시스템 프롬프트 — 모델의 역할을 고정
•SYSTEM """너는 서울시 민원 상담 전문가야.
•답변 규칙:
•1. 3줄 이내로 답한다
•2. 반드시 존댓말을 쓴다
•3. 관련 법령이 있으면 번호를 포함한다
•4. 모르는 것은 "확인 후 안내드리겠습니다"라고 한다"""

•# 하이퍼파라미터 설정
•PARAMETER temperature 0.3
•PARAMETER top_p 0.9
•PARAMETER top_k 40
•PARAMETER num_predict 512
•PARAMETER repeat_penalty 1.1
•PARAMETER num_ctx 4096
•PARAMETER stop "<|end|>"
•PARAMETER stop "<|user|>"
```

- ollama create 민원봇 -f 민원봇.Modelfile
- ollama run 민원봇

Ollama에서 하이퍼파라미터를 조정하는 3가지 방법

방법 1: 대화 중 /set 명령어 (가장 간편)

```
ollama run gemma4:e4b
>>> /set parameter temperature 0.1
>>> 다음 민원을 분류하라: "골목에 쓰레기가 방치되어 악취가 심합니다"

>>> /set parameter temperature 1.2
>>> 다음 민원을 분류하라: "골목에 쓰레기가 방치되어 악취가 심합니다"
```

```
/set parameter temperature 0.3
/set parameter top_p 0.9
/set parameter top_k 40
/set parameter num_predict 512
/set parameter repeat_penalty 1.1
/set parameter seed 42
/set parameter num_ctx 4096
/set system "너는 서울시 민원 상담 전문가야. 3줄 이내로 존댓말로 답한다."
```

방법 2: Modelfile로 커스텀 모델 생성 (업무별 고정 설정)

```
FROM gemma4:e4b

SYSTEM """너는 서울시 민원 상담 전문가야.
답변 규칙:
1. 3줄 이내로 답한다
2. 반드시 존댓말을 쓴다
3. 관련 법령이 있으면 번호를 포함한다"""

PARAMETER temperature 0.3
PARAMETER top_p 0.9
PARAMETER num_predict 512
PARAMETER num_ctx 4096
PARAMETER seed 42
```

```
ollama create 민원봇 -f 민원봇.Modelfile
ollama run 민원봇
```

방법3: API 호출 시 options에 포함 (파이썬·HTML에서)

```
import requestsresponse =
requests.post('http://localhost:11434/api/chat', json={
    "model": "gemma4:e4b",
    "messages": [
        {"role": "user", "content": "민원을 분류하라: 골목에 쓰레기가 방
치됨"}    ],
    "options": {
        "temperature": 0.1,
        "top_p": 0.9,
        "num_predict": 256,
        "seed": 42    }})
print(response.json()['message']['content'])
```

M6. Ollama + 레거시 폐쇄망

Hugging Face + Ollama 조합

Hugging Face에서 GGUF 형식 모델을 다운로드하면 Ollama에서 바로 쓸 수 있다.

- # Hugging Face에서 GGUF 모델 직접 가져오기
- ollama create my-model -f Modelfile

- FROM hf.co/bartowski/gemma-2-2b-it-GGUF:Q4_K_M
- SYSTEM "너는 한국어 민원 상담 전문가야"
- PARAMETER temperature 0.3

Hugging Face에서 직접 가져오기

GGUF 파일을 수동으로 다운받을 필요 없이 Hugging Face 경로를 직접 지정할 수 있다:

bash
GGUF 모델 직접 가져오기 ollama create my-model -f Modelfile

Modelfile에서:
FROM hf.co/bartowski/Phi-3-mini-4k-instruct-GGUF:Q4_K_M

또는 Safetensors 모델도:
FROM hf.co/microsoft/phi-3-mini-4k-instruct

Safetensors면 Ollama가 알아서 GGUF로 변환한 뒤 등록한다. 시간이 좀 더 걸릴 뿐.

활용	방법	인터넷 필요	난이도
데모 체험	Spaces 웹에서 클릭	필요	쉬움
감성 분석	transformers pipeline	최초 1회만	중
텍스트 요약	transformers pipeline	최초 1회만	중
제로샷 분류	transformers pipeline	최초 1회만	중
질의응답 (추출형)	transformers pipeline	최초 1회만	중
OCR	transformers pipeline	최초 1회만	중
음성→텍스트	whisper pipeline	최초 1회만	중
웹 UI 붙이기	Gradio	불필요 (로컬)	중
Ollama에 모델 추가	GGUF 다운로드	최초 1회만	쉬움
학습 데이터 확보	datasets 라이브러리	최초 1회만	쉬움

M6. Ollama + 레거시 폐쇄망

Ollama 활용 실습 — 레거시 폐쇄망 시나리오

[워크플로우 개요]

Ollama 개념·모델·Modelfile·Hugging Face 조합을 실제 부서 업무에 끼워 넣는 실습이다. Ollama는 단독으로 쓰기보다 이미 부서가 쓰는 레거시 도구(엑셀·워드·HTML 단일파일)와 결합할 때 진가를 발휘한다. 외부 LLM API에 데이터를 보낼 수 없는 폐쇄망 부서가 「그래도 LLM의 힘은 빌리고 싶다」는 요구를 충족시키는 가장 현실적인 답이다. 본 실습은 부서 PC에 Ollama를 설치하고 작은 모델(1B~4B)을 받은 뒤, 엑셀 셀에서 호출하거나 단일 HTML에서 호출하는 식으로 활용한다. 모든 데이터가 부서 PC 안에서만 이동한다 — 외부망 차단 환경에서도 LLM이 작동하는 첫 경험이 된다.

#	시나리오	결합돋구	입력	출력
A	엑셀 셀에서 Ollama로 텍스트 요약	엑셀 VBA	셀의 긴 텍스트 (회의록·민원 내용)	옆 셀에 3줄 요약
B	엑셀 셀에서 Ollama로 분류	엑셀 VBA	민원 내용	카테고리 자동 분류
C	단일 HTML에서 Ollama 챗봇 만들기	HTML/CSS/JS + fetch	사용자 질문	부서 맞춤 응답
D	워드 문서 어투·맞춤법 교정 매크로	워드 VBA	문서의 선택 영역	교정된 텍스트로 치환
E	HTML 단일파일로 부서 문서 자가진단	HTML + Ollama	작성한 보고서 텍스트	가이드라인 적합성 점검 결과
F	GGUF 모델 받아 부서 맞춤 응답	Ollama + Modelfile	부서 도메인 질문	시스템 프롬프트로 튜닝된 응답

「외부망 없이 LLM이 작동한다」 부서 PC 안에서 GPT 비슷한 것이 돌아가고, 엑셀이 그것을 호출한다. 데이터는 한 발자국도 부서 밖으로 나가지 않는다. **폐쇄망 환경에서 가능한 가장 강력한 자동화 — Ollama × 레거시 결합.**

- Ollama가 작동 중인지 확인 (ollama list로 받은 모델 목록 확인)
- LLM에 「Ollama API를 호출하는 VBA/JavaScript 코드」 요청 — 호출 주소(http://localhost:11434/api/chat), 사용할 모델명, 시스템 프롬프트 명시
- 받은 코드를 엑셀(VBA) 또는 HTML 파일에 붙여넣기
- 작은 입력으로 먼저 검증 → 응답 확인 → 본격 사용
- 응답 품질 부족하면 Modelfile로 시스템 프롬프트·하이퍼파라미터 조정

- 응답이 너무 느림 → 더 작은 모델(llama3.2:1b 또는 qwen3.5:0.8b)로 변경
- 응답이 영어로 나옴 → 시스템 프롬프트에 「반드시 한국어로 답하라」 명시
- 응답 품질이 낮음 → Modelfile로 temperature 0.3, top_p 0.9 등 조정
- 엑셀이 응답 기다리는 동안 멈춤 → 비동기 호출 또는 Application.ScreenUpdating = False
- 한글 인코딩 깨짐 → JSON 요청·응답 모두 UTF-8 명시
- Ollama 서버가 가끔 끊김 → 자동 재시도 로직 추가

M7 — 클라우드 노코드 풀스택 도구 비교

M7 — 클라우드 노코드 풀스택 도구 비교

바이브 코딩이란 무엇인가

바이브코딩의 정의와 그 유형

바이브 코딩(Vibe Coding)이라는 용어는 2025년 2월 앤드류 카파시 (전 테슬라 AI 디렉터, OpenAI 공동창립자)가 처음 썼다. 그의 정의는 이렇다: "코드를 직접 쓰지 않고, AI에게 자연어로 지시해서 코드를 생성하고, 결과물의 '느낌(vibe)'이 맞으면 그대로 쓰는 프로그래밍 방식."

> •기존 프로그래밍: 사람이 코드를 쓴다 → 컴퓨터가 실행한다
> •바이브 코딩: 사람이 말한다 → AI가 코드를 쓴다 → 컴퓨터가 실행한다 → 사람이 결과를 보고 "이거 맞아/아니야"를 판단한다

- **UI 지향형 —** 예쁜 화면(프론트엔드)을 빠르게 만드는 데 특화. 디자인 품질이 높고, 컴포넌트 단위로 세밀하게 조정 가능. 대신 백엔드(서버·DB)는 약하거나 없음.
- **풀스택 지향형 —** 프론트엔드+백엔드+데이터베이스를 한 번에 생성. MVP(최소 기능 제품)를 빠르게 만들 수 있지만, 세부 디자인 품질은 UI 지향형보다 떨어질 수 있음.
- **데모/프로토타입 지향형 —** 아이디어를 빠르게 시각화하는 것이 목적. 완성도보다 속도 우선.
- **에이전트 지향형 —** AI가 코드를 쓰고, 실행하고, 테스트하고, 배포까지 자율적으로 수행

도구	가격	유형	강점	약점	공무원 적합도
Lovable	무료(5크레딧/일) / $20/월	UI+풀스택	디자인 완성도 최상. Supabase+자체 백엔드. 보안 감사($100) 제공. ARR $100M(8개월)	무료 제한적. 복잡 로직은 반복 수정 필요	★★★★★
Bolt.new	무료(1M토큰/월) / $25/월	풀스택	가장 빠른 프로토타이핑. 브라우저에서 완결. Netlify 배포 내장	코드 품질 낮음(프로토타입용). 백엔드 약함	★★★★☆
v0 (Vercel)	무료(제한) / $20/월	UI 컴포넌트	개별 UI 컴포넌트 디자인 최고 품질. React/Tailwind. Figma 연동	전체 앱 생성보다 컴포넌트 특화. 백엔드 없음	★★★☆☆
Replit Agent	무료(제한) / $25/월	에이전트+풀스택	설치 제로. 50+ 언어. 실시간 협업. 배포 원스톱	무료 컴퓨팅 제한. AI가 의도와 다르게 수정하는 경우	★★★★☆
Base44	무료(5크레딧/일) / 유료	내부 도구 특화	데이터 모델+UI 동시 설계. CRUD 앱에 최적. 노코드에 가장 가까움	외부 공개용 디자인 부족. 커뮤니티 작음	★★★★☆
Google AI Studio	무료	AI 실험	Gemini API 무료 실험. 멀티모달. 프롬프트→코드 자동 변환	웹앱 직접 생성 불가. UI 안 나옴	★★★☆☆
Firebase Studio	무료	풀스택+배포	Google 인프라. Gemini 내장. Firebase 배포 즉시. 앱 프로토타이핑	구글 생태계 종속. 학습 곡선 있음	★★★☆☆
Trae (ByteDance)	무료 (얼리 액세스)	AI IDE	Claude Sonnet + GPT-4o 무료 포함. VS Code 포크. 완전 무료	얼리 액세스 불안정. 장기 가격 불확실	★★★☆☆
A0.dev	유료	모바일 앱	React Native 네이티브 모바일 앱 생성. iOS/Android 동시	웹 전용이 아닌 모바일 특화. 비용 높음	★★☆☆☆

M7 — 클라우드 노코드 풀스택 도구 비교

코딩에이전트

코딩 에이전트/AI IDE (개발자용 — 기존 코드베이스에서 작업)

도구	가격	유형	강점	공무원 적합도
Cursor	무료(제한) / $20/월	AI IDE (VS Code 포크)	자동완성 최고 속도. 멀티파일 편집. Agent 모드. 유저 100만+	★★☆☆☆
Windsurf	무료(제한) / $20/월	AI IDE (VS Code 포크)	Cascade 지속 맥락. SWE-1.5 모델. Cursor보다 자율성 높음	★★☆☆☆
Antigravity (Google)	무료 (프리뷰)	AI IDE	멀티에이전트 병렬 작업. 브라우저 내장. Mission Control	★★★☆☆
Claude Code	API 종량제 / Max $100/월	CLI	1M 토큰 컨텍스트. 추론 최강. 에러 자동 수정 루프	★★★☆☆
OpenAI Codex	ChatGPT Plus $20/월 포함	CLI+클라우드+데스크톱	클라우드 샌드박스. 추론 강도 조절. Issue→PR 자동	★★★☆☆
OpenCode	무료 (오픈소스 MIT)	CLI	75+ 모델 자유 연결. GitHub 스타 14만+. 오프라인 가능	★★★☆☆
Gemini CLI	무료	CLI	구글 계정만으로 프론티어 모델 무료 사용. 1M 컨텍스트	★★★☆☆
GitHub Copilot	무료(제한) / $10/월	IDE 확장	1,500만 유저. 모든 IDE 지원. 가장 저렴. Agent 모드	★★☆☆☆
Aider	무료 (오픈소스)	CLI	Git 네이티브. 커밋 자동 생성. 가볍고 빠름	★★☆☆☆
Devin	$20/월 + ACU 종량	클라우드	"AI 소프트웨어 엔지니어". 가장 자율적. PR 머지율 67%	★★★★☆

M7 — 클라우드 노코드 풀스택 도구 비교

노코드 서비스 활용 실습

[워크플로우 개요]

C05에서 도구 지형(Lovable·Bolt·v0·Replit·Base44·AI Studio 등)을 살펴봤다면, C06 M7은 「같은 요구사항을 두 도구에 동시에 던져 결과를 비교하는」 손작업이다. 표만 봐서는 어느 도구가 본인 업무에 맞는지 감이 잡히지 않는다. 직접 제작·비교를 한 번 해봐야 도구 선택 기준이 몸에 새겨진다. 본 실습은 부서 단위 풀스택 서비스 한 가지를 정해 두 도구에 같은 프롬프트로 던지고, 결과(완성도·디자인·DB 자동연결·확장성)를 매트릭스로 정리한다. 이 비교가 다음 모듈 M8(Lovable + Supabase 깊이)의 도구 선택 근거가 된다.

#	시나리오	결합돋구	입력	출력
A	부서 게시판 (글·댓글·검색)	회원 가입 + 글 작성 + 댓글 + 검색	Lovable vs Bolt	★
B	결재 신청 양식 (DB 저장·승인 흐름)	신청 → DB 저장 → 승인 상태 변경	Lovable vs Replit	★★
C	일정·회의실 예약 시스템	캘린더 UI + 예약 등록 + 충돌 검증	Lovable vs Base44	★★
D	UI 컴포넌트 vs 풀스택 비교	같은 디자인을 v0와 Lovable에 동시	v0 vs Lovable	★★
E	6개 도구 미니 제로샷 (재현)	간단한 부서 도구 1개	수강자 선택한 도구 6개	★★★

「표 보고 고르는 도구」가 아니라 「직접 만들어 보고 고르는 도구」. 같은 요구사항을 두 도구에 던져 보면 표에서 안 보이던 차이가 한 번에 드러난다. 5분의 비교가 다음 모듈 M8에서 쓸 도구 선택의 가장 강력한 근거가 된다.

- 본인 부서 시나리오 1개 확정 (위 풀에서 선택 또는 부서 실제 페인포인트)
- 같은 요구사항 프롬프트 작성 (두 도구에 동일하게 입력할 텍스트)
- 도구 1에 입력 → 결과 받기 → 5분 내 마이너 보정 1회
- 도구 2에 동일 프롬프트 입력 → 결과 받기 → 5분 내 마이너 보정 1회
- 두 결과를 비교 매트릭스에 정리 (아래 항목)
- 본인 부서에 더 적합한 도구 1개 결정 (M8에서 본격 사용할 도구 선정 근거)

- 응답이 너무 느림 → 더 작은 모델(llama3.2:1b 또는 qwen3.5:0.8b)로 변경
- 응답이 영어로 나옴 → 시스템 프롬프트에 「반드시 한국어로 답하라」 명시
- 응답 품질이 낮음 → Modelfile로 temperature 0.3, top_p 0.9 등 조정
- 엑셀이 응답 기다리는 동안 멈춤 → 비동기 호출 또는 Application.ScreenUpdating = False
- 한글 인코딩 깨짐 → JSON 요청·응답 모두 UTF-8 명시
- Ollama 서버가 가끔 끊김 → 자동 재시도 로직 추가

M8. Lovable + Supabase 자동연동

M8. Lovable + Supabase 자동연동

Lovable이란 무엇인가

Lovable

자연어로 프롬프트만 던지면 프론트엔드·백엔드·DB·인증·배포까지 한 번에 만들어주는 풀스택 AI 코딩 플랫폼이다.

2025년 2월 출시 이후 AI 코딩 도구 중 가장 빠르게 성장. 2026년 현재 비개발자가 풀스택 웹앱을 만드는 사실상의 표준 도구로 자리잡음.

DIFFERENTIATION

기존 노코드와 무엇이 다른가

- **풀스택 자동 생성**
  화면·로직·DB·인증을 한 번의 프롬프트로 동시에 만든다.
- **Lovable Cloud 내장 (Supabase 기반)**
  DB·인증·Storage·Edge Functions가 별도 설정 없이 즉시 작동.
- **Lovable AI 내장**
  GPT-5.2·Gemini 3 Flash 사용. API 키 관리 불필요. 이미지 생성 포함.
- **즉시 배포 + GitHub 자동 동기화**
  Lovable 도메인 1클릭 게시. Custom Domain·GitHub 양방향 연동.

HOW IT WORKS

프롬프트 한 번 → 작동하는 풀스택

1. **자연어 요구사항 입력**
   "부서 게시판을 만들어줘. 회원·글·댓글·검색 포함."
2. **React 프론트 + Supabase DB 자동 생성**
   테이블 설계·RLS·인증까지 1분 내 구성
3. **실시간 미리보기 + Visual Edits 보정**
   화면 클릭으로 즉석 편집 (Figma 스타일)
4. **Publish — 공개 URL 또는 Custom Domain**
   동료에게 링크 전달, 동시 GitHub 푸시

PRICING (2026 기준)

플랜	월 가격	크레딧	주요 기능
Free	$0	5/일 · 30/월	학습·체험용
Pro	$25	100/월	Private 프로젝트·Custom Domain
Business	$50+	사용량 기반	팀 공유·SSO·데이터 프라이버시

CORE INSIGHT

Lovable의 본질은 「풀스택을 한 묶음으로 자동화」 — DB·인증·AI·배포가 별도 설정 없이 한 번에 묶여 나온다.

M8. Lovable + Supabase 자동연동

Lovable Cloud — 백엔드 자동화

Lovable Cloud

Lovable이 내부적으로 구동하는 Supabase 기반 백엔드다.
사용자는 Supabase 계정·콘솔을 만질 필요가 없다.

"이 앱에 회원가입·로그인 추가해줘", "댓글 데이터를 저장해줘"라고 말하면
Lovable이 알아서 테이블을 만들고, 인증 흐름을 붙이고, RLS 정책까지 설정한다.

HOW TO ENABLE

활성화는 한 번의 클릭

프로젝트에서 「Cloud」 토글을 ON으로 바꾸면 끝.
또는 Lovable에 "회원가입 기능을 만들어줘"라고 말하면
알아서 Cloud를 활성화하고 인증 흐름을 구성한다.

PRICING

월 $25 사용량까지 무료 — 초기 개발·테스트 충분

사용자 트래픽이 증가하면 사용량 기반 과금. 인스턴스 업그레이드 가능.

4 BUILT-IN FEATURES

01 · AUTH

인증·권한

- Email·Password 가입·로그인
- Google·Apple OAuth (자동 생성)
- Row-Level Security 자동 설정
- 2FA 지원

02 · DATABASE

데이터베이스

- PostgreSQL 기반 (Supabase)
- 테이블·관계 자동 설계
- 자동 마이그레이션
- Realtime 동기화 가능

03 · STORAGE

파일 저장소

- 이미지·문서·동영상 업로드
- 공개·비공개 버킷 분리
- 자동 CDN 배포
- 권한별 접근 제어

04 · EDGE FUNCTIONS

서버사이드 로직

- 서버리스 함수 자동 배포
- 외부 API·결제·이메일 결합
- Auth 검증 자동
- 로그·재시도 자동 처리

+ EXTENDED INTEGRATIONS

LLM · 이미지 생성 · 결제 · 이메일까지 한 묶음

Lovable AI 활성화 시 GPT-5.2·Gemini 3·Nano Banana 호출, Stripe·Resend 같은 외부 서비스도 프롬프트로 결합 가능.

CORE INSIGHT

「Supabase를 직접 만지지 않고도 Supabase의 모든 기능을 쓴다」 — 비개발자의 백엔드 진입 장벽이 무너진 지점.

M8. Lovable + Supabase 자동연동

Lovable AI — LLM·이미지 자동 통합

Lovable AI

"이 앱에 요약 기능 넣어줘", "이미지 생성 기능 추가해줘"라고 말하면 끝.
API 키 발급·관리 없이 GPT·Gemini를 즉시 호출. 사용량은 Lovable 빌링에 통합되어 청구된다.

SUPPORTED MODELS

모델	제공사	특징	상태
Gemini 3 Flash	Google	빠름·저비용·기본 모델	DEFAULT
GPT-5.2	OpenAI	추론 강력·복잡한 작업 적합	선택 가능
Nano Banana	Google	이미지 생성 (아바타·배경·이미지 자산)	선택 가능

HOW TO ENABLE

활성화는 두 가지 방법

방법 1 · 프로젝트 설정에서 「Lovable AI」 토글 ON
방법 2 · "AI 기능을 추가해줘"라고 말하면 알아서 활성화
활성화 시 Cloud도 함께 활성화됨 (백엔드 함수에서 LLM 호출하는 구조).

USE CASES IN PUBLIC SECTOR

CASE 01
민원 자동 분류
민원 입력 → LLM이 부서·우선순위 분류

CASE 02
회의록 자동 요약
긴 회의록 입력 → 핵심 안건·결정사항 요약

CASE 03
정책 문서 챗봇
부서 자료 업로드 → 자연어 질의응답

CASE 04
이미지 자동 생성
홍보 이미지·다이어그램·아이콘 즉석 제작

PROMPT EXAMPLE

"민원 게시판에 새 글이 등록되면 LLM이 자동으로 부서를 분류해서 라벨로 붙여줘. 모델은 Gemini 3 Flash 사용."

CORE INSIGHT

API 키 관리·토큰 계산·재시도 로직 — 모두 사라진다. "AI 기능 넣어줘" 한 줄이면 끝.

M8. Lovable + Supabase 자동연동

작업 모드 · Visual Edits · Knowledge

TWO WORKING MODES

두 가지 작업 모드

DEFAULT MODE

코드 생성·수정

자연어 요청 → AI가 코드를 직접 작성·수정·실행한다.

적합한 작업
새 페이지 추가, 기능 확장, 버그 수정, 디자인 변경

CHAT MODE

대화·디버깅·기획

코드를 쓰지 않고 대화만 한다. 설계·문제 분석·아이디어 정리.

적합한 작업
기능 설계 토론, 에러 원인 추적, 기획 단계 브레인스토밍

VISUAL EDITS

Figma 스타일 비주얼 편집

미리보기 화면에서 요소를 직접 클릭해서 색·크기·텍스트를 즉시 수정.
"이 버튼 빨강으로 바꿔줘"라고 말하지 않아도 클릭으로 끝낸다.

언제 쓰나
색·정렬·여백 같은 미세 디자인 보정. 프롬프트보다 빠르고 정확하다.

CUSTOM KNOWLEDGE

프로젝트 컨텍스트 기억

프로젝트 설정에 부서 정보·브랜딩·코딩 컨벤션·양식 규칙을 적어두면 Lovable이 모든 편집에서 그것을 기억하고 일관성 있게 따른다.

예시 적용
- "부서 색은 #003366 네이비, 모든 헤더는 굵게"
- "민원 분류는 4개 카테고리만 사용 (시민·교통·환경·복지)"

BOOKMARK · HISTORY

버전 복원

중요한 시점을 북마크로 저장. 잘못된 변경을 한 번에 되돌릴 수 있다.

언제 쓰나
대규모 리팩토링 직전, 실험적 기능 추가 직전

LABS

실험적 기능

설정 → Labs에서 베타 기능을 활성화하고 미리 사용 가능.

최근 추가
- Chat-Only Mode
- Test/Live 환경 분리
- OAuth 자동 통합

CORE INSIGHT

「프롬프트 + 비주얼 편집 + 컨텍스트 기억」— 세 가지가 결합되어 일관된 결과물을 빠르게 만든다.

M8. Lovable + Supabase 자동연동

외부 도구 연동 — Lovable의 확장성

Lovable은 닫힌 도구가 아니다. 디자인·코드·도메인·외부 서비스로 자유롭게 확장된다.
"내가 만든 결과물을 어디로든 가져갈 수 있다"가 Lovable의 또 다른 강점.

01 · DESIGN

Figma → 코드 변환

Figma 디자인 파일을 그대로 Lovable에 가져오면 작동하는 React 컴포넌트로 변환된다.

활용 시점
디자이너가 만든 부서 페이지 시안을 즉석 구현으로 전환

02 · CODE

GitHub 양방향 동기화

Lovable에서 만든 코드는 그대로 GitHub 저장소로 푸시되고, 로컬에서 수정한 코드도 다시 Lovable로 동기화된다.

활용 시점
개발자와 협업 · 부서 인계 · 버전 관리 · 코드 영구 보관

03 · DOMAIN

Custom Domain · CDN

Pro 플랜에서 본인 도메인 연결. Cloudflare·CloudFront 같은 CDN·프록시도 거쳐 운영 가능.

활용 시점
기관 도메인 하위 서비스 운영
예: app.부서.go.kr

04 · SERVICES

외부 서비스 결합

결제·이메일·알림·SMS·이미지·웹 스크래핑까지 프롬프트로 결합 가능.

결합 가능 도구
Stripe · Resend · Twilio · Replicate · Firecrawl 등

05 · REMIX (프로젝트 복제)

잘 만든 프로젝트를 한 번 복제해서 다양하게 변주

한 번 잘 만든 「부서 게시판」 프로젝트를 Remix하면, 그것을 출발점으로 「민원 게시판」, 「자료 공유 게시판」, 「업무 공지판」을 빠르게 파생시킬 수 있다.
부서 표준 양식을 한 번 잘 만들어 두고, 모든 산하 팀이 그것을 Remix해서 자기 용도로 변형 — Lovable이 부서 단위 자산이 되는 패턴.
※ Shopify 같은 일부 외부 연동이 있는 프로젝트는 Remix 불가 (해당 연동을 끊은 후 가능)

CORE INSIGHT

「Lovable에 갇히지 않는다」 — 디자인은 Figma에서, 코드는 GitHub에서, 운영은 본인 도메인에서. 자유롭게 들고 나갈 수 있다.

M8. Lovable + Supabase 자동연동

실습 시나리오 풀

시나리오 중 1~2개를 선택하여 실습. 모든 시나리오는 인증·DB·Auth·배포가 결합된 완성형 풀스택을 목표로 한다.
실습 시간 90분 — 시나리오 1개 30~40분이 표준. 시간 여유 있을 때 두 번째 시나리오로 변형 응용.

SCENARIO POOL

#	시나리오	핵심 기능	활용 Lovable 기능	난이도
A	**부서 게시판**	회원가입 · 글 · 댓글 · 검색 · 카테고리	Cloud (DB·Auth) · Visual Edits	★
B	**결재 신청 양식**	신청 → DB 저장 → 승인 흐름 → 상태 변경	Cloud (RLS) · Edge Functions	★★
C	**일정·회의실 예약 시스템**	캘린더 UI · 예약 등록 · 충돌 검증	Cloud · Realtime · Custom Knowledge	★★
D	**부서 자료 라이브러리 + 챗봇**	자료 업로드 · 분류 · 검색 · LLM 질의응답	Cloud · Storage · Lovable AI	★★★
E	**민원 자동 분류·접수 시스템**	민원 입력 → LLM 자동 분류 → 부서 라벨링 → 알림	Cloud · Lovable AI (Gemini 3) · Auth	★★★
F	**부서 설문조사 · 응답 통계**	설문 작성 · 응답 수집 · 차트 · CSV 내보내기	Cloud · Edge Functions	★★

WORKFLOW

실습 진행 흐름

- 시나리오 선택 → 1페이지 요구사항 작성
- Lovable에 자연어 프롬프트 입력 → 첫 결과 받기
- Cloud 활성화 → DB·Auth 자동 구성 확인
- Visual Edits·프롬프트로 보정 → Publish 또는 GitHub 푸시

M9. 디자인 → 코드 파이프라인 (Stitch)

M9. 디자인 → 코드 파이프라인 (Stitch)

Google Stitch란 무엇인가

Stitch

자연어·이미지·음성으로 UI 디자인을 즉석에서 생성하는 Google Labs의 AI 디자인 도구다.

2025년 5월 출시 후 2026년 3월 「Stitch 2.0」 업데이트로 본격 개편. 단일 화면 생성 도구에서 「Vibe Design」 캔버스 기반 멀티스크린 디자인 환경으로 진화.

DIFFERENTIATION

Figma·기존 디자인 도구와 다른 점

- **Vibe Design — 와이어프레임 단계 생략**
  목적·감정·영감 이미지만 던지면 바로 high-fidelity UI 생성
- **멀티스크린 일괄 생성**
  한 프롬프트로 최대 5개 연결된 화면을 일관된 디자인 시스템으로
- **코드 export 내장**
  HTML·CSS·Tailwind·React로 즉시 변환 → AI Studio·Figma·Antigravity 전송
- **무료 (Google Labs)**
  월 350회 표준 + 200회 실험 생성 제공

HOW IT WORKS

자연어 → UI → 코드

1. **의도·맥락 입력**
   "민원 접수 화면을 만들고 싶다. 친근한 톤, 단계별 진행"
2. **Gemini가 멀티스크린 생성**
   최대 5개 화면을 일관된 디자인 시스템으로 동시에
3. **Direct Edit·Voice로 즉석 수정**
   컴포넌트 클릭, 음성 명령으로 변형·재구성
4. **코드 export → 다른 도구로 이동**
   AI Studio·Figma·Antigravity·MCP로 핸드오프

MODELS · PRICING

Standard Mode
Gemini 3 Flash · 빠른 생성
월 350회 무료
일상 디자인·빠른 탐색용

Thinking Mode
Gemini 3.1 Pro · 깊은 추론
월 200회 (Low/Med/High)
복잡한 멀티스크린 흐름용

CORE INSIGHT

Stitch는 「디자인 단계의 Lovable」 — 자연어로 화면을 만들고, 만든 결과를 Lovable·Antigravity·Figma로 그대로 넘긴다.

M9. 디자인 → 코드 파이프라인 (Stitch)

Stitch 2.0 — 핵심 5가지 기능

2026년 3월 발표된 Stitch 2.0은 단일 화면 생성기에서 「디자인 작업 환경」으로 진화했다.
아래 5가지 기능이 Stitch 2.0의 정체성을 이룬다.

01 · VIBE DESIGN

와이어프레임 단계 생략

기존: 와이어프레임 → 시안 → 고품질 디자인
Stitch: 의도·감정·영감만 던져도 즉시 high-fidelity

예시 프롬프트
"민원 접수 앱을 만들고 싶어. 친근하고 단계별로 진행되는 느낌. 색은 따뜻한 톤."

02 · INFINITE CANVAS

AI 네이티브 무한 캔버스

생성된 모든 디자인 변형이 캔버스에 누적
덮어쓰기 없이 비교·선택·조합 가능

강점
"세 번째 안의 헤더 + 첫 번째 안의 버튼"처럼 여러 시안의 일부를 자유롭게 결합

03 · MULTI-SCREEN

멀티스크린 일괄 생성

한 프롬프트로 최대 5개 연결 화면 생성
색·타이포·컴포넌트가 일관된 디자인 시스템

예시
"민원 접수 흐름 — 입력 / 확인 / 처리 진행 / 완료 / 후속 안내 5단계 화면 만들어줘"

04 · VOICE DESIGN MODE

음성 디자인 — 캔버스에 말하기

Stitch가 디자인 에이전트로 작동. 음성 명령에 실시간 반응한다.

활용 예시
"이 메뉴를 세 가지 다른 안으로 보여줘" · "다른 색 팔레트로 시안 만들어줘"
"이 흐름의 단점을 비평해줘" — AI가 즉석 비평·재안 제안

05 · DIRECT EDIT · INSTANT PROTOTYPE

컴포넌트 클릭으로 즉석 편집·연결

개별 요소 클릭 → 4가지 액션 (수정·다음 화면 상상·복제·삭제)

Instant Prototype
화면 우클릭 → 「Instant Prototype」 → 화면들이 자동으로 연결되어 실제 흐름처럼 클릭으로 이동 가능. 별도 와이어링 불필요.

CORE INSIGHT

Stitch 2.0의 본질 = 「디자이너의 손작업」을 「AI와의 대화」로. 여러 시안을 동시에 보고, 비교하고, 합친다.

M9. 디자인 → 코드 파이프라인 (Stitch)

디자인 시스템 · Export · 실습

DESIGN.MD — DESIGN SYSTEM

디자인 시스템을 마크다운으로

Stitch는 디자인 시스템(색·폰트·간격·컴포넌트)을 「DESIGN.md」 한 파일로 관리한다. 프로젝트 간 가져오기·내보내기 가능, 오픈소스 사양으로 공개됨.

활용 시점

- 부서 표준 색·폰트·여백 규칙을 한 번 만들어 모든 화면에 일관 적용
- 기관 브랜드 가이드라인을 DESIGN.md로 옮겨 두면 모든 디자인이 자동 준수
- WCAG 접근성 검증까지 AI가 자동으로 (각 색상의 용도가 명시되어 있어)

EXPORT — 다른 도구로 보내기

Stitch에서 만든 디자인은 어디로든

- **Figma — Auto Layout 유지하며 편집 가능한 레이어로**
- **AI Studio — Gemini 기반 코드·앱으로 즉시 변환**
- **Antigravity — 코딩 에이전트가 풀스택으로 구현**
- **Firebase Studio — Google 풀스택 환경으로 핸드오프**
- **MCP / .zip / HTML·CSS·React 코드 — 자유 export**

PRACTICE SCENARIOS

실습 시나리오 — 디자인 → 코드

#	시나리오	난이도
A	민원 접수 흐름 5화면 → AI Studio로 코드	★
B	부서 대시보드 → Lovable로 풀스택 구현	★★
C	사진 영감 업로드 → 디자인 변환 → Figma export	★
D	기관 가이드라인 DESIGN.md 작성 → 화면 일괄 생성	★★★
E	음성으로 디자인 비평·재안 받기 (Voice Mode)	★★

WORKFLOW

실습 진행 흐름

- 의도·맥락 자연어로 입력 → 첫 시안 생성
- Direct Edit·Voice로 시안 보정 → Infinite Canvas에서 비교
- 만족스러운 시안 선택 → Instant Prototype으로 흐름 연결
- Export → AI Studio·Lovable·Antigravity로 코드 변환

PIPELINE

Stitch (디자인 생성) → AI Studio·Antigravity·Lovable (코드 변환) → 작동하는 서비스. 디자이너 없이도 부서 단위 풀스택 흐름.

M10. 인증·권한 기초

M10. 인증·권한 기초 (Lovable Cloud)

Lovable Cloud로 인증·권한 다루기

Supabase 콘솔을 직접 만지지 않고도 기관급 인증·권한 시스템을 구축한다.
"이 페이지는 로그인한 사람만", "이 데이터는 본인 것만 보이게" — 자연어로 요청하면 Lovable이 인증 흐름과 RLS를 자동으로 설계한다.

01 · EMAIL · PASSWORD

기본 가입·로그인

이메일·비밀번호 기반 가입·로그인.
비밀번호 재설정·이메일 확인 자동.

자동 구성 항목
- 가입 폼 · 로그인 폼
- 비밀번호 재설정 흐름

02 · GOOGLE · APPLE OAUTH

소셜 로그인

"Google·Apple 로그인 추가해줘"
한 마디로 OAuth 흐름 자동 생성.

강점
- 가입 마찰 최소화
- 비밀번호 관리 부담 제거

03 · ROW-LEVEL SECURITY

행 단위 권한 제어

PostgreSQL의 RLS 정책을
Lovable이 자동으로 작성한다.

예시 자연어 요청
"본인이 작성한 글만 수정·삭제
가능하게 해줘"

04 · ROLES · PERMISSIONS

역할·권한 분리

관리자·일반·승인자 등
역할별 접근 범위 설정.

예시
- 일반: 신청서 작성·조회
- 승인자: 승인·반려 가능

PROMPT PATTERNS — 자연어로 인증 구축

기본 인증 추가
"이 앱에 회원가입·로그인 기능 추가해줘. 이메일·비밀번호 기반."

소셜 로그인 추가
"Google·Apple 로그인 옵션 추가해줘."

권한별 페이지 분리
"관리자 페이지를 만들고 admin 역할만 접근 가능하게 해줘."

행 단위 권한
"각 사용자는 본인이 작성한 글만 수정·삭제 가능하게 RLS 설정해줘."

CORE INSIGHT

「Auth는 어렵다」는 건 옛말. Lovable Cloud가 OAuth·RLS·역할 관리를 자연어 한 줄로 처리한다.

M10. 인증·권한 기초 (Lovable Cloud)

보안 강화 + 실습 시나리오

SECURITY · 2FA

2단계 인증 (2FA)

Authenticator 앱 또는 SMS 기반 추가 인증 단계.
민감 데이터를 다루는 부서 서비스에 권장.

자연어 요청
"관리자 계정에는 2FA 의무화해줘.
Authenticator 앱 기반."

ENTERPRISE SECURITY

기관급 보안 옵션 (Business 플랜)

- **SSO (Single Sign-On)**
  기관 인증 시스템과 연동 — Workspace 단위 일괄 로그인
- **그룹 기반 접근 제어**
  부서별·직급별 접근 권한을 그룹 단위로 관리
- **데이터 프라이버시 컨트롤**
  개인정보 분리·암호화·접근 로그 기록

※ 일반 부서 학습용은 Free·Pro로 충분. SSO·고급 보안은 Business 이상 (월 $50+)

PRACTICE SCENARIOS

인증·권한 실습 시나리오

#	시나리오	난이도
A	M8 게시판에 회원가입·로그인 추가	★
B	Google·Apple 소셜 로그인 추가	★
C	관리자·일반 페이지 분리 (역할 기반)	★★
D	RLS — 본인 글만 수정·삭제 가능	★★
E	결재 시스템 — 신청자·승인자 권한 분리	★★★

WORKFLOW

실습 진행 흐름

- M8에서 만든 풀스택 앱 열기 → 인증 추가 자연어 요청
- 가입·로그인 화면 자동 생성 확인
- 테스트 계정 2개 생성 → 권한 차이 작동 확인
- RLS·역할 분리 추가 → 보안 검증

CORE INSIGHT

서비스 운영의 진짜 시작은 「누가, 무엇을, 언제 볼 수 있는가」 — 인증·권한이 잡혀야 부서 도구가 된다.

M11. GitHub + Netlify/Vercel 배포

M11. GitHub + Netlify/Vercel 배포

결과물을 세상에 공개하는 두 가지 길

본 과정에서 만든 결과물은 두 가지 경로로 공개된다.
결과물 종류에 따라 적절한 경로를 선택한다. 어느 쪽이든 「공개 URL → 동료에게 전달」이 목표다.

PATH 01

Lovable Publish

M8에서 만든 Lovable 프로젝트 → 1클릭 배포

대상 결과물
Lovable로 만든 풀스택 웹앱

배포 흐름
- 우측 상단 「Publish」 버튼 클릭
- 자동 부여 도메인 (your-app.lovable.app) 생성
- 또는 Custom Domain 연결 (Pro 이상)
- 새 배포 = 1클릭 (코드 푸시·CI 빌드 모두 자동)

강점
- 별도 GitHub·CI 설정 불필요
- Cloud (DB·Auth) 자동 연결 유지
- Cloudflare·CloudFront 같은 CDN·프록시 통과 가능

PATH 02

GitHub → Netlify · Vercel

단일 HTML 또는 코딩에이전트 결과물 → 무료 정적 호스팅

대상 결과물
M3 단일 HTML 도구 / 코딩에이전트로 만든 정적 사이트

배포 흐름
- GitHub에 저장소 만들고 파일 업로드
- Netlify 또는 Vercel에서 GitHub 저장소 연결
- 자동 빌드·배포 → 공개 URL 생성
- Git에 푸시할 때마다 자동 재배포 (CI/CD)

강점
- 완전 무료 (트래픽 한도 충분)
- 단일파일·정적 사이트에 최적
- Custom Domain·HTTPS 자동 적용

CORE INSIGHT

「내 PC에서만 작동」 → 「공개 URL로 누구나 접근」 — 도구가 부서 자산으로 격상되는 마지막 한 발자국.

M11. GitHub + Netlify/Vercel 배포

GitHub → Netlify·Vercel 배포 단계

STEP 01 · GITHUB

GitHub 저장소에 올리기

코드와 파일을 GitHub에 보관하는 단계.
한 번 올리면 백업·이력관리·협업이 모두 따라온다.

단계

- github.com에서 New Repository 생성 (Public 또는 Private)
- 결과물 파일을 「Add file → Upload files」로 드래그
- Commit 메시지 한 줄 적고 저장
- 명령창 익숙하면 git init·push 명령으로도 동일 가능

STEP 02 · NETLIFY · VERCEL

정적 호스팅에 연결

GitHub 저장소를 Netlify·Vercel에 연결하면
자동 빌드·배포·HTTPS·도메인까지 한 번에.

단계

- netlify.com 또는 vercel.com에 GitHub 계정으로 가입
- 「Import Project」 → GitHub 저장소 선택
- 자동 빌드 후 공개 URL 생성 (예: my-tool.netlify.app)
- 이후 Git push할 때마다 자동 재배포

TIP — LLM에 도움 받기

GitHub도 LLM 어시스트로

"GitHub 저장소를 처음 만들어 파일을 올리고 싶어. Windows 환경, 명령창 사용. 단계별로 알려줘"라고 LLM에 요청하면 그대로 따라하면 된다.

자주 쓰는 명령

git init · git add . · git commit -m "메시지" · git push

에러 메시지가 나오면 그대로 LLM에 붙여 다음 단계 안내 받기

NETLIFY vs VERCEL

어느 것을 쓸까

Netlify

- 드래그앤드롭 배포 가능 (GitHub 없어도 즉시)
- 단일 HTML·정적 사이트에 강함

Vercel

- React·Next.js에 최적화
- 코딩에이전트 결과물 배포에 적합

SUMMARY

Lovable 결과물 → Lovable Publish 1클릭. 단일 HTML·코딩에이전트 결과물 → GitHub → Netlify/Vercel.

M12. 종합 워크숍 — 전체 종합 실습